

事象駆動型分散安全強化学習に基づく 複数ドローンによる協調取り囲み実験

2026 年 5 月 29 日

概要

本稿では、SMC Otani 論文 [1] の事象駆動型コンセンサス primal-dual 安全強化学習アルゴリズムを、複数の DJI Tello ドローンによる移動対象物の協調取り囲み実験へ移植するための問題設定、定式化、アルゴリズム、および実装方法をまとめる。取り囲み制御は逆運動学に基づく協調取り囲み問題 [2] を参考にし、対象物を中心とする正多角形配置、および対象物の進行方向に応じた配置変形を扱う。学習問題は、各 Tello を 1 エージェントとする制約付きマルコフ決定過程 (CMDP) として記述し、目的関数には取り囲み誤差、フォーメーション誤差、逆運動学参照速度からのずれ、制御入力の大きさを入れ、制約関数には機体間衝突、対象物への過接近、飛行領域逸脱、高度逸脱、入力飽和を入れる。特に、Tello SDK 2.0 User Guide [3] に記載されている `command`, `takeoff`, `land`, `stop`, `rc a b c d`, `go x y z speed` などの簡易テキストコマンドだけを用いて協調取り囲みを実装できるように、低レベルの推力・姿勢角入力を仮定しない SDK コマンド層を明示する。一方で、本稿の分散性は、主として方策パラメータと制約乗数の更新・通信が通信グラフ上の近傍交換により行われるという意味である。Unity 数値実験および実機初期検証では、目的コスト・制約コスト・安全シールドを外部計測 PC または Unity 環境が全状態から評価する centralized training/decentralized execution (CTDE) 型の構成を許す。さらに、本稿の Tello/Unity 実装は、Otani 論文の exact-gradient 理論問題をそのまま実現するものではなく、有限ホライズンのロールアウトに基づく局所 surrogate gradient 実装として位置付け、SDK コマンド制約下での取り囲み性能、empirical constraint cost, およびイベントトリガ通信削減を実験的に評価する。

目次

1	研究テーマの狙いと全体像	3
2	記号と座標系	4
3	DJI Tello のダイナミクスとパラメータ	5
3.1	6 自由度剛体モデル	5
3.2	実装用の SDK コマンドレベルモデル	8
3.3	SDK 簡易コマンドだけで協調取り囲みを実装する考え方	9
4	協調取り囲み制御	11
4.1	正多角形による取り囲み目標	11
4.2	対象物の進行方向に応じた配置変形	11
4.3	逆運動学に基づく参照速度	13

5	CMDP と制約付き方策探索としての定式化	16
5.1	状態, 観測, 行動	16
5.2	目的コスト	18
5.3	安全制約コスト	19
5.4	制約付き方策探索問題	20
6	各エージェントが持つ変数と交換する情報	22
7	事象駆動型コンセンサス primal-dual 学習アルゴリズム	24
7.1	重み行列とイベントトリガ	24
7.2	ロールアウト勾配推定	24
7.3	提案アルゴリズムの疑似コード	26
8	数値実験設定	27
8.1	共通設定	27
8.2	実験 1: 静止対象物の正多角形取り囲み	28
8.3	実験 2: 等速移動対象物の進行方向依存取り囲み	28
8.4	実験 3: 円軌道・折れ線軌道の対象物取り囲み	28
8.5	実験 4: 通信削減効果の評価	29
8.6	実験 5: Tello ダイナミクス・遅延・ノイズへの頑健性	30
9	Unity と Python による数値実験の実現方法	30
9.1	推奨ソフトウェア構成	30
9.2	Unity 側のクラス設計	31
9.3	Unity 1 ステップの処理	31
9.4	Python 側の学習ループ	31
9.5	ログと図の出力	32
10	DJI Tello 実機実験の実現方法	33
10.1	実機実験での基本方針	33
10.2	推奨ハードウェア構成	33
10.3	実機ソフトウェア構成	34
10.4	SDK コマンド層の実機実装	34
10.5	実機制御ループ	36
10.6	安全シールドの例	36
10.7	段階的な実機移行手順	38
11	実装時に特に注意すべき点	38
11.1	Otani 論文の理論保証と本実験の関係	38
11.2	分散性の説明	38
11.3	Tello 実機での通信と制御周期	39

1 研究テーマの狙いと全体像

本研究の目的は、対象物の移動に追従しながら、複数の小型ドローンが安全に協調取り囲みを学習することである。具体的には、 N 台の DJI Tello を用い、各 Tello が自分の局所観測と近傍エージェントから受け取る限定的な情報に基づいて行動する。全体としては、対象物の周囲に半径 R の正多角形状フォーメーションを形成し、対象物が移動する場合には、文献 [2] の「進行方向を意識した取り囲み配置」に近い形へ変形する。

SMC Otani 論文 [1] のアルゴリズムは、分散安全強化学習を bounded multipliers を用いた compact-dual min-max surrogate problem として扱い、各エージェントが局所方策パラメータと制約乗数スタックの局所コピーをもち、近傍から最後に受信した値だけを使ってコンセンサス型の primal-dual 更新を行うものである。重要な点は、毎回通信するのではなく、**現在値と最後に送信した値の差がしきい値を超えた場合だけ送信することである**。本稿では、この仕組みを Tello ドローンの協調取り囲み問題へ実験用に移植する。ただし、後述するように、本稿の Unity/Tello 実験は有限ホライズンのロールアウト勾配を用いる実装版であり、Otani 論文の exact-gradient 理論問題そのものではない。したがって、本稿で扱う実験は、**Otani 論文の定理を実機上で直接検証するものではなく、その事象駆動型 consensus primal-dual 構造を Tello の SDK コマンド制約下で動作するロールアウト型安全 RL 実装へ落とし込むための実験設計である**。

本稿でいう分散性は、主として方策パラメータ ϑ_i と制約乗数 μ_i の更新・通信が通信グラフ $\mathcal{G}_{\text{comm}}$ 上の近傍交換だけで行われるという意味である。一方、Unity 数値実験および実機初期検証では、安全確保と再現性のため、外部計測系または Unity 環境が全状態を観測し、目的コスト・制約コスト・安全シールドを評価する CTDE 型構成を採用する。このため、本稿は「学習更新の近傍通信」と「訓練時・実機安全監視の中央評価インフラ」を明確に分けて記述する。

本稿の主な貢献は次の四点である。第一に、Tello SDK の rc, go, stop, takeoff, land だけを用いる command-level abstraction を与え、推力・姿勢角・モータ速度を直接入力とする非現実的な実装仮定を避ける。第二に、対象物取り囲みを CTDE 型 CMDP として定式化し、正多角形配置、進行方向依存配置、逆運動学参照速度、安全制約コストを同一の実験設計内で扱う。第三に、イベントトリガ通信が、取り囲み性能と empirical constraint cost を大きく損なわずに方策・乗数通信を削減できるかを評価する指標を整理する。第四に、Unity から Tello 実機へ段階的に移行するため、raw action, shielded action, SDK コマンド、安全シールド、緊急停止条件を分離した実装手順を与える。

本稿で考える構成は次の通りである。

- **エージェント**: 各 DJI Tello ドローン $i \in \{1, 2, \dots, N\}$.
- **環境**: 屋内飛行領域、移動対象物、障害物、壁、床、外部計測系、または Unity シミュレータ.
- **各エージェントの学習変数**: 方策パラメータ $\vartheta_{i,q}$, 価値関数パラメータ $\xi_{i,q}$, 制約乗数の局所コピー $\mu_{i,q}$, 最後に送信した値 $\hat{\vartheta}_{i,q}$, $\hat{\mu}_{i,q}$.
- **各エージェントが交換する情報**: 学習イテレーション q においてイベントが発生したときのみ、 $\hat{\vartheta}_{i,q}$ と $\hat{\mu}_{i,q}$ を通信グラフ上の近傍に送る。実機制御で近傍相対位置が必要な場合は、外部計測系から全

表 1 本研究における三つの既存要素の役割

既存要素	本研究での使い方
分散安全強化学習 [1]	更新則として用いる．各 Tello は局所方策パラメータ ϑ_i と制約乗数スタック μ_i をもち，イベントトリガにより必要なときだけ ϑ_i, μ_i を近傍へ送る．
取り囲み制御 [2]	逆運動学参照速度，評価指標，および比較ベースラインとして用いる．特に，対象物周りの正多角形配置と，対象物の進行方向に応じた配置変形を用いる．
DJI Tello SDK と機体モデル [3, 4]	実機ではロータ推力や姿勢角を直接指令せず，rc, go, stop, takeoff, land などの SDK テキストコマンドだけを使う．6 自由度モデルは Unity 内の近似モデル・domain randomization・パラメータ設計に用いる．

エージェントに配布するか，別途，低レートの状態共有を行う．

- **マルチエージェント系の目的:** 対象物を正多角形状または進行方向依存の形で取り囲み，かつ衝突・壁接近・高度逸脱・入力飽和などを抑える．
- **安全制約:** 強化学習上の制約コストとして扱うものと，実機では外側の安全シールドでリアルタイムに抑制するものを分ける．後者は計測遅延，速度飽和，QP の可解性，非常停止系の作動を前提とする工学的な防護層として扱う．

2 記号と座標系

エージェント集合を $\mathcal{V} = \{1, 2, \dots, N\}$ とする．本稿では，**通信グラフ**と**フォーメーショングラフ**を分けて考える．通信グラフを無向グラフ

$$\mathcal{G}_{\text{comm}} = (\mathcal{V}, \mathcal{E}_{\text{comm}})$$

とし， $\mathcal{N}_i^{\text{comm}}$ をエージェント i の通信近傍集合とする．方策パラメータと制約乗数のイベントトリガ通信は，この通信グラフ上でのみ行う．一方，正多角形や進行方向依存配置の辺長誤差を定義するためのフォーメーショングラフを

$$\mathcal{G}_{\text{form}} = (\mathcal{V}, \mathcal{E}_{\text{form}})$$

と書く．本稿の基本設定では $\mathcal{G}_{\text{form}}$ はサイクルグラフであり， i^- と i^+ はフォーメーショングラフ上の前後隣接エージェントを表す．通信グラフをサイクル以外に変える場合でも，フォーメーション誤差の定義には $\mathcal{G}_{\text{form}}$ を用いる．学習イテレーションを $q = 0, 1, 2, \dots$ とし，1 エピソード内の制御ステップを $k = 0, 1, \dots, H - 1$ とする．また，制御周期を Δt とする．

Tello のモデルでは，慣性座標系を $\{E\}$ ，機体座標系を $\{B_i\}$ とする．Giernacki らの Tello モデル [4] は NED 系で記述されることがあるが，Unity や一般的な屋内ロボット実験では ENU 系や Unity の Y 軸上向き座標も使われる．本稿では，後続の式 (7) の重力・推力の符号と整合させるため，**数式上の 6 自由度モデルは z 軸上向きの ENU 型座標**として記述する．NED 系の実装やログを用いる場合は， z 座標と高度の符号を変換してから本稿の式に代入する．

- 数式上の高忠実度 6 自由度モデルでは, $\mathbf{p}_i = [x_i, y_i, z_i]^T$ を ENU 型慣性座標系の位置, $\boldsymbol{\eta}_i = [\phi_i, \theta_i, \psi_i]^T$ を roll, pitch, yaw の Euler 角とする. また, 機体座標系角速度を $\boldsymbol{\omega}_i^B = [\omega_{i,x}^B, \omega_{i,y}^B, \omega_{i,z}^B]^T$ と書く. Euler 角速度 $\dot{\boldsymbol{\eta}}_i$ と機体角速度 $\boldsymbol{\omega}_i^B$ は一般には一致しないため, 角運動方程式では両者を分けて扱う.
- 取り囲み制御と強化学習の観測は, 実装しやすいように水平面座標 $\mathbf{r}_i = [x_i, y_i]^T$ と高度 $h_i = z_i$ を用いて書く. 実機ログが NED 系で得られる場合は $h_i = -z_i^{\text{NED}}$ と読み替える.
- Unity では, Unity 座標 $\mathbf{p}_i^U = [X_i, Y_i, Z_i]^T$ を用い, 水平面を (X, Z) , 高度を Y とする. 数式の水平面座標とは $\mathbf{r}_i = [X_i, Z_i]^T$, 高度 $h_i = Y_i$ で対応させる.

対象物の水平面位置を $\mathbf{r}_{o,k} \in \mathbb{R}^2$, 速度を $\mathbf{v}_{o,k} \in \mathbb{R}^2$ とする. ドローン i の水平面位置, 水平速度, 高度, yaw 角をそれぞれ

$$\mathbf{r}_{i,k} \in \mathbb{R}^2, \quad \mathbf{v}_{i,k} \in \mathbb{R}^2, \quad h_{i,k} \in \mathbb{R}, \quad \psi_{i,k} \in \mathbb{R}$$

とする. 取り囲み半径を $R > 0$, 目標高度を $h^{\text{ref}} > 0$ とする. 高度誤差の正規化には, 実機の物理的な高度上限と混同しないよう, スケール定数 $h_{\text{scale}} > 0$ を用いる. 実機安全シールドで使う高度上限・下限は, 後述の $h_{\text{max}}^{\text{real}}, h_{\text{min}}^{\text{real}}$ として別に表す.

本稿では, 機体姿勢角の pitch を θ_i , roll を ϕ_i と書くため, 強化学習の方策パラメータには ϑ , 価値関数パラメータには ξ を用いる. これにより, Tello の物理姿勢角と学習パラメータの記号衝突を避ける. また, R は取り囲み半径として用い, ステップ目的コストは後で ℓ_i と書く.

観測正規化と実機安全シールドに使うスケール定数は, 本文中で初出する前に次の意味で用いる. R_{max} は観測内の相対位置を無次元化する代表長, v_{max} は水平速度正規化用の代表速度, $v_{o,\text{max}}$ は対象物速度正規化用の代表速度, $v_{z,\text{max}}$ は上下速度正規化用の代表速度, $d_{\text{max}}^{\text{wall}}$ は壁距離特微量の正規化長である. $b_{i,k}$ は Tello i のバッテリー残量, $\mathcal{B}_k$ はバッテリー・通信品質・計測欠落などの実験状態, $\mathcal{O}_{\text{obs}}$ は障害物集合を表す. 実機安全シールドで用いる $d_{\text{emergency}}, u_{\text{max}}^{\text{real}}, u_{z,\text{max}}^{\text{real}}, u_{\text{dead}}, u_{\text{rc}}^{\text{max,real}}, \alpha_h, \alpha_w, \alpha_z, \rho_\delta, \rho_z$ は, 10 節で定義する安全監視用パラメータであり, 学習上の平均制約しきい値 d_i とは区別する.

3 DJI Tello のダイナミクスとパラメータ

3.1 6 自由度剛体モデル

Giernacki らは, Tello を重心に質量が集中した剛体とし, 4 つのロータを X 型配置としてモデル化している [4]. 以下では, Unity 実装と水平面取り囲み記法に合わせて, z 軸上向きの ENU 型座標で記述する. NED 系で実装する場合は, 位置・速度・重力・推力方向の符号を対応する座標変換でそろえる. 機体座標系から慣性座標系への回転行列を

$$Q_i = R_Z(\psi_i)R_Y(\theta_i)R_X(\phi_i) \quad (1)$$

とする。ただし,

$$R_X(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \quad (2)$$

$$R_Y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad (3)$$

$$R_Z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (4)$$

である。ここで, $c_\phi = \cos \phi$, $s_\phi = \sin \phi$ とすると,

$$Q_i = \begin{bmatrix} c_{\psi_i} c_{\theta_i} & c_{\psi_i} s_{\theta_i} s_{\phi_i} - s_{\psi_i} c_{\phi_i} & c_{\psi_i} s_{\theta_i} c_{\phi_i} + s_{\psi_i} s_{\phi_i} \\ s_{\psi_i} c_{\theta_i} & s_{\psi_i} s_{\theta_i} s_{\phi_i} + c_{\psi_i} c_{\phi_i} & s_{\psi_i} s_{\theta_i} c_{\phi_i} - c_{\psi_i} s_{\phi_i} \\ -s_{\theta_i} & c_{\theta_i} s_{\phi_i} & c_{\theta_i} c_{\phi_i} \end{bmatrix} \quad (5)$$

となる。

ロータ角速度を $\omega_{i,1}, \omega_{i,2}, \omega_{i,3}, \omega_{i,4}$ とし, 総推力を

$$T_i = b \sum_{r=1}^4 \omega_{i,r}^2 \quad (6)$$

とする。 z 軸上向きの ENU 型座標では, 位置の運動方程式は

$$\ddot{\mathbf{p}}_i = -g \mathbf{e}_3 + Q_i \frac{b}{m} \sum_{r=1}^4 \omega_{i,r}^2 \mathbf{e}_3 - \frac{1}{m} \begin{bmatrix} k_1 \dot{x}_i \\ k_2 \dot{y}_i \\ k_3 \dot{z}_i \end{bmatrix} \quad (7)$$

である。ここで $\mathbf{e}_3 = [0, 0, 1]^\top$, m は質量, k_1, k_2, k_3 は並進抗力係数である。成分表示すると

$$\ddot{x}_i = (\sin \phi_i \sin \psi_i + \sin \theta_i \cos \phi_i \cos \psi_i) \frac{u_{1,i}}{m} - \frac{k_1}{m} \dot{x}_i \quad (8)$$

$$\ddot{y}_i = (-\sin \phi_i \cos \psi_i + \sin \theta_i \cos \phi_i \sin \psi_i) \frac{u_{1,i}}{m} - \frac{k_2}{m} \dot{y}_i \quad (9)$$

$$\ddot{z}_i = -g + \cos \phi_i \cos \theta_i \frac{u_{1,i}}{m} - \frac{k_3}{m} \dot{z}_i \quad (10)$$

となる。ただし, $u_{1,i} = T_i$ である。

角運動では, Euler 角 $\boldsymbol{\eta}_i = [\phi_i, \theta_i, \psi_i]^\top$ と機体座標系角速度 $\boldsymbol{\omega}_i^B$ を区別する。標準的な剛体近似として,

$$I \dot{\boldsymbol{\omega}}_i^B = -\boldsymbol{\omega}_i^B \times I \boldsymbol{\omega}_i^B - I_R (\boldsymbol{\omega}_i^B \times \mathbf{e}_3) \Omega_{R,i} + \begin{bmatrix} u_{2,i} \\ u_{3,i} \\ u_{4,i} \end{bmatrix} - \mathbf{d}_\omega(\boldsymbol{\omega}_i^B) \quad (11)$$

を用いる。ここで,

$$\Omega_{R,i} = \sum_{r=1}^4 \sigma_r \omega_{i,r}, \quad (\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (-1, +1, -1, +1) \quad (12)$$

であり, σ_r は各ロータの回転方向を表す符号である。クアッドロータでは隣接ロータが反対方向に回転するため, ロータジャイロ項には単純な $\sum_{r=1}^4 \omega_{i,r}$ ではなく, 回転方向を含む $\Omega_{R,i}$ を用いる。また,

表 2 DJI Tello 論文に記載された Tello の同定パラメータ

記号	意味	値	単位
m	機体質量	0.08	kg
l	ロータ軸から重心までの距離	0.06	m
I_{xx}, I_{yy}	慣性モーメント	0.679×10^{-2}	kg m ²
I_{zz}	慣性モーメント	1.313×10^{-2}	kg m ²
I_R	各ロータの慣性モーメント	4.95×10^{-5}	kg m ²
b	推力定数	0.566×10^{-5}	kg m/rad ²
d	ドラッグ係数	0.762×10^{-7}	kg m ² /rad ²
k_1, k_2, k_3	並進抗力係数	3.365×10^{-2}	kg/s
k_4, k_5, k_6	空力摩擦係数	4.609×10^{-3}	kg m ² /rad
T_m	モータ応答時定数	0.0163	s

$u_{2,i}, u_{3,i}, u_{4,i}$ は roll, pitch, yaw 方向のトルクであり, $\mathbf{d}_\omega$ は角速度依存の抵抗項である. 例えば, 表 2 の k_4, k_5, k_6 を使う場合は, $\mathbf{d}_\omega(\boldsymbol{\omega}_i^B) = [k_4\omega_{i,x}^B|\omega_{i,x}^B|, k_5\omega_{i,y}^B|\omega_{i,y}^B|, k_6\omega_{i,z}^B|\omega_{i,z}^B|]^\top$ のように実装できる. Euler 角の時間発展は

$$\dot{\boldsymbol{\eta}}_i = T_\eta(\boldsymbol{\eta}_i)\boldsymbol{\omega}_i^B, \quad T_\eta(\boldsymbol{\eta}_i) = \begin{bmatrix} 1 & \sin \phi_i \tan \theta_i & \cos \phi_i \tan \theta_i \\ 0 & \cos \phi_i & -\sin \phi_i \\ 0 & \sin \phi_i / \cos \theta_i & \cos \phi_i / \cos \theta_i \end{bmatrix} \quad (13)$$

で与える. このように書くと, Euler 角速度をそのまま剛体角速度として用いる不整合を避けられる.

X 型配置のミキサは

$$\begin{bmatrix} u_{1,i} \\ u_{2,i} \\ u_{3,i} \\ u_{4,i} \end{bmatrix} = \begin{bmatrix} b & b & b & b \\ \frac{\sqrt{2}}{2}lb & -\frac{\sqrt{2}}{2}lb & -\frac{\sqrt{2}}{2}lb & \frac{\sqrt{2}}{2}lb \\ \frac{\sqrt{2}}{2}lb & \frac{\sqrt{2}}{2}lb & -\frac{\sqrt{2}}{2}lb & -\frac{\sqrt{2}}{2}lb \\ -d & d & -d & d \end{bmatrix} \begin{bmatrix} \omega_{i,1}^2 \\ \omega_{i,2}^2 \\ \omega_{i,3}^2 \\ \omega_{i,4}^2 \end{bmatrix} \quad (14)$$

となる. さらに, モータ応答の一次遅れを入れる場合は

$$\dot{\omega}_{i,r} = \frac{1}{T_m} (\omega_{i,r}^{\text{cmd}} - \omega_{i,r}), \quad r = 1, 2, 3, 4 \quad (15)$$

を用いる.

■注意. 表 2 の慣性モーメント値は Giernacki ら [4] に記載された値をそのまま用いる. ただし, $m = 0.08$ kg, 代表長さ $l = 0.06$ m から得られる粗いスケール $ml^2 = 2.88 \times 10^{-4}$ kg m² と比べると, 表 2 の 10^{-2} オーダーの値は大きく見える. そのため, 実機同定を新たに行う場合や SI 単位での厳密な高忠実度シミュレーションを行う場合には, 原典の単位, 指数表記, および座標系を必ず再確認する. 特に, 本稿の式は ENU 型の z 軸上向き座標で書いているため, NED 系の既存コードや同定表を流用する場合は, 重力方向・推力方向・高度の符号を必ず変換する. また, 実際の DJI Tello ではロータ角速度, 総推力, roll/pitch 角を直接指令できず, Tello SDK のテキストコマンドを UDP で送る形になる [3]. したがって, (7)–(14) は Unity 内の高忠実度近似モデルまたは制御器設計用モデルとして使い, 実機では次節のコマンドレベルモデルを主に使うのが安全である.

3.2 実装用の SDK コマンドレベルモデル

実機 Tello は内部の姿勢安定化・速度制御を閉じたブラックボックスとしてもち、外部 PC からは Tello SDK/UDP または ROS を介して、あらかじめ定義されたテキストコマンドだけを送る [3, 4]. したがって、本稿の実装では、低レベル入力 u_1, u_2, u_3, u_4 や目標姿勢角を直接指令するのではなく、SDK の rc a b c d または go x y z speed へ変換できる抽象行動を使う. 強化学習と Unity 実験では、まず rc コマンドに対応する次のコマンドレベルモデルを基本モデルとして使う. エージェント i の行動を

$$\mathbf{a}_{i,k} = [a_{i,k}^{\text{lr}}, a_{i,k}^{\text{fb}}, a_{i,k}^{\text{ud}}, a_{i,k}^{\text{yaw}}]^T \in [-1, 1]^4 \quad (16)$$

とする. ここで lr は左右, fb は前後, ud は上下, yaw は yaw 角速度である. 実機では

$$\mathbf{u}_{i,k}^{\text{rc}} = \text{round}(100\mathbf{a}_{i,k}) \in \{-100, \dots, 100\}^4 \quad (17)$$

を rc a b c d の 4 チャンネルに変換して Tello に送る. この rc は Tello SDK に用意されている遠隔操縦用の set command であり、成分は left/right, forward/backward, up/down, yaw の整数値 $[-100, 100]$ である [3]. Unity では、安全のため実機より小さい最大速度を設定する.

$$v_{i,x,k}^{B,\text{cmd}} = v_{\text{fb}}^{\text{max}} a_{i,k}^{\text{fb}}, \quad v_{i,y,k}^{B,\text{cmd}} = v_{\text{lr}}^{\text{max}} a_{i,k}^{\text{lr}}, \quad (18)$$

$$v_{i,z,k}^{\text{cmd}} = v_{\text{ud}}^{\text{max}} a_{i,k}^{\text{ud}}, \quad \dot{\psi}_{i,k}^{\text{cmd}} = \omega_{\psi}^{\text{max}} a_{i,k}^{\text{yaw}}. \quad (19)$$

機体座標系の前後・左右速度を水平面の世界座標へ変換すると

$$\begin{bmatrix} v_{i,x,k}^{\text{cmd}} \\ v_{i,y,k}^{\text{cmd}} \end{bmatrix} = \begin{bmatrix} \cos \psi_{i,k} & -\sin \psi_{i,k} \\ \sin \psi_{i,k} & \cos \psi_{i,k} \end{bmatrix} \begin{bmatrix} v_{i,x,k}^{B,\text{cmd}} \\ v_{i,y,k}^{B,\text{cmd}} \end{bmatrix} \quad (20)$$

となる. ここでは、 $a_{i,k}^{\text{fb}} > 0$ を機体前方、 $a_{i,k}^{\text{lr}} > 0$ を機体左方向に対応させている. Tello SDK や利用する Python/ROS ライブラリが左右方向を右向き正として扱う場合は、RC 指令へ変換する直前で $a_{i,k}^{\text{lr}}$ の符号を反転する. この符号規約を **Unity, Python, 実機ドライバで必ず統一する**.

速度応答は一次遅れで近似する.

$$\mathbf{v}_{i,k+1} = \mathbf{v}_{i,k} + \frac{\Delta t}{\tau_v} (\mathbf{v}_{i,k}^{\text{cmd}} - \mathbf{v}_{i,k}) + \Delta t \mathbf{w}_{i,k}^v, \quad (21)$$

$$\mathbf{r}_{i,k+1} = \mathbf{r}_{i,k} + \Delta t \mathbf{v}_{i,k+1}, \quad (22)$$

$$\dot{h}_{i,k+1} = \dot{h}_{i,k} + \frac{\Delta t}{\tau_z} (v_{i,z,k}^{\text{cmd}} - \dot{h}_{i,k}) + \Delta t w_{i,k}^h, \quad (23)$$

$$h_{i,k+1} = h_{i,k} + \Delta t \dot{h}_{i,k+1}, \quad (24)$$

$$\dot{\psi}_{i,k+1} = \dot{\psi}_{i,k} + \frac{\Delta t}{\tau_{\psi}} (\dot{\psi}_{i,k}^{\text{cmd}} - \dot{\psi}_{i,k}) + \Delta t w_{i,k}^{\psi}, \quad (25)$$

$$\psi_{i,k+1} = \psi_{i,k} + \Delta t \dot{\psi}_{i,k+1}. \quad (26)$$

ここで $\mathbf{w}^v, w^h, w^{\psi}$ は風や通信遅延・内部制御誤差を模擬するノイズである. 初期値としては

$$\Delta t = 0.1 \text{ s}, \quad \tau_v = 0.25 \text{ s}, \quad \tau_z = 0.30 \text{ s}, \quad (27)$$

$$\tau_{\psi} = 0.20 \text{ s}, \quad v_{\text{fb}}^{\text{max}} = v_{\text{lr}}^{\text{max}} = 0.4 \text{ m/s}, \quad v_{\text{ud}}^{\text{max}} = 0.25 \text{ m/s}, \quad (28)$$

$$\omega_{\psi}^{\text{max}} = 60^\circ/\text{s}. \quad (29)$$

程度から始めると、屋内実機実験へ移行しやすい。一次遅れモデルを陽的 Euler 法で実装する場合は、少なくとも $0 < \Delta t/\tau_v < 1$, $0 < \Delta t/\tau_z < 1$, $0 < \Delta t/\tau_\psi < 1$ となる設定から始めると数値的に安定しやすい。Tello の仕様上はより大きな速度が可能だが、実験室内の協調取り囲みでは、まず低速で安全に動かすべきである。実機では RC 指令値 $[-100, 100]$ と実速度の対応は機体差、バッテリー残量、床面テクスチャ、照明条件、内部制御状態に依存するため、単機で複数の RC 指令を与え、外部計測系で平均速度を測定して $\mathbf{a} \mapsto \mathbf{v}^{\text{cmd}}$ のスケールを事前にキャリブレーションする。

3.3 SDK 簡易コマンドだけで協調取り囲みを実装する考え方

Tello SDK 2.0 では、PC から Tello へ UDP で文字列コマンドを送る。最初に `command` を送って SDK mode に入り、飛行開始は `takeoff`、終了は `land`、緊急停止は `emergency` を送る。状態値は別ポートで受信でき、姿勢角、速度、高度、バッテリー残量などが文字列として返る [3]。この仕様は一見単純だが、協調取り囲みに必要な「各時刻で少しずつ位置を補正する」動作は、`rc a b c d` を短周期で送り続けることで実装できる。低レベルの推力や姿勢角を直接使う必要はない。

表 3 本稿の協調取り囲みで使う Tello SDK コマンドの役割

SDK コマンド	協調取り囲みでの使い方
<code>command</code>	各機を SDK mode に入れる初期化コマンド。これを送る前に他の飛行コマンドを送らない。
<code>takeoff, land</code>	実験開始・終了のためのコマンド。学習や取り囲み制御の連続入力ではなく、状態遷移として扱う。
<code>stop</code>	空中で停止・ホバリングさせるコマンド。安全シールドが危険を検出したとき、または waypoint 誤差が十分小さいときに使う。
<code>rc a b c d</code>	推奨する主制御コマンド。4 チャンネルの整数値で左右、前後、上下、yaw を同時に指定できるため、協調取り囲みのフィードバック制御に最も適している。
<code>go x y z speed</code>	低レート of waypoint 追従モードに使う。各成分は cm, speed は cm/s で指定する。小さすぎる移動量は SDK 仕様上受け付けられないため、滑らかな閉ループ補正には <code>rc</code> の方が扱いやすい。
<code>forward, back, left, right, up, down, cw, ccw</code>	初学者向けの単機動作確認、座標系・符号確認、デバッグに使う。協調取り囲み本番では、これらを直接 RL action にするより、 <code>rc</code> または <code>go</code> へ変換する方が安全である。

■推奨実装 A：rc による短周期フィードバック。協調取り囲みでは、各 Tello が自分のスロットへ近づきつつ、機体間安全距離と壁距離を守る必要がある。このような連続的な微修正には、`rc a b c d` を $T_{\text{rc}} = 0.05\text{--}0.1$ s 程度で繰り返し送る実装が最も自然である。方策または NSB 参照速度から得た世界座標系の望ましい水平速度を $\mathbf{u}_{i,k}^{xy,\text{des}} \in \mathbb{R}^2$ とする。例えば、学習を使わないデバッグ段階では

$$\mathbf{u}_{i,k}^{xy,\text{des}} = \mathbf{v}_{o,k} + K_p(\mathbf{r}_{i,k}^{\text{slot}} - \mathbf{r}_{i,k}) + K_d(\mathbf{v}_{i,k}^{\text{slot}} - \mathbf{v}_{i,k}) \quad (30)$$

のように置けばよい。学習済み方策を使う場合は、方策出力 $\mathbf{a}_{i,k}^{\text{RL}}$ を式 (18) で速度へ変換し、安全シールド後の速度を $\mathbf{u}_{i,k}^{xy,\text{safe}}$ とする。いずれの場合も、最後は世界座標系速度を Tello 機体座標系の前後・左右成分へ変換する。

$$\begin{bmatrix} v_{i,k}^{B,\text{fb}} \\ v_{i,k}^{B,\text{left}} \end{bmatrix} = \begin{bmatrix} \cos \psi_{i,k} & \sin \psi_{i,k} \\ -\sin \psi_{i,k} & \cos \psi_{i,k} \end{bmatrix} \mathbf{u}_{i,k}^{xy,\text{safe}}. \quad (31)$$

左右チャンネルの符号はライブラリにより「右を正」としている場合と「左を正」としている場合があるため、符号係数 $\sigma_{\text{lr}}^{\text{SDK}} \in \{-1, +1\}$ を単機キャリブレーションで決める。送信する整数コマンドは

$$u_{i,k}^{\text{rc,lr}} = \text{clip}_{[-100,100]} \left(\text{round} \left(100 \sigma_{\text{lr}}^{\text{SDK}} \frac{v_{i,k}^{B,\text{left}}}{v_{\text{lr}}^{\text{max}}} \right) \right), \quad (32)$$

$$u_{i,k}^{\text{rc,fb}} = \text{clip}_{[-100,100]} \left(\text{round} \left(100 \frac{v_{i,k}^{B,\text{fb}}}{v_{\text{fb}}^{\text{max}}} \right) \right), \quad (33)$$

$$u_{i,k}^{\text{rc,ud}} = \text{clip}_{[-100,100]} \left(\text{round} \left(100 \frac{u_{i,k}^{z,\text{safe}}}{v_{\text{ud}}^{\text{max}}} \right) \right), \quad (34)$$

$$u_{i,k}^{\text{rc,yaw}} = \text{clip}_{[-100,100]} \left(\text{round} \left(100 \frac{\omega_{i,k}^{\psi,\text{safe}}}{\omega_{\psi}^{\text{max}}} \right) \right) \quad (35)$$

となる。そして、Tello Driver は

$$\text{rc } u_{i,k}^{\text{rc,lr}} \ u_{i,k}^{\text{rc,fb}} \ u_{i,k}^{\text{rc,ud}} \ u_{i,k}^{\text{rc,yaw}}$$

を送る。目標位置近傍で停止したいときは、`stop` または `rc 0 0 0 0` を送る。Tello は一定時間コマンドを受信しないと自動着陸する仕様を持つため [3]、制御ループが一時停止した場合にも、安全が確認できる範囲では `keep-alive` として `rc 0 0 0 0` または `stop` を送る設計にする。

■実装 B：go による低レート waypoint 追従。rc を使わず、より「命令列」に近い実装にしたい場合は、`go x y z speed` による低レート waypoint 追従を使う。この場合、制御周期ごとに細かい速度を送るのではなく、 $T_{\text{wp}} = 0.5\text{--}1.0$ s ごとに次の短距離 waypoint を作る。例えば、

$$\mathbf{p}_{i,k}^{\text{wp}} = \begin{bmatrix} \mathbf{r}_{i,k}^{\text{slot}} \\ h^{\text{ref}} \end{bmatrix} + T_{\text{look}} \begin{bmatrix} \mathbf{v}_{i,k}^{\text{ref,clip}} \\ 0 \end{bmatrix}, \quad \Delta \mathbf{p}_{i,k}^E = \mathbf{p}_{i,k}^{\text{wp}} - \mathbf{p}_{i,k} \quad (36)$$

を計算し、 $\Delta \mathbf{p}_{i,k}^E$ を機体座標系に回転して cm 単位へ変換する。水平面の変換は式 (31) と同じである。得られた body-frame の増分を整数 cm 単位に丸めて

$$x_i^{\text{go}} = \text{clip}_{[-500,500]} \left(\text{round} \left(100 \Delta p_{i,k}^{B,\text{fb}} \right) \right), \quad (37)$$

$$y_i^{\text{go}} = \text{clip}_{[-500,500]} \left(\text{round} \left(100 \sigma_{\text{lr}}^{\text{go}} \Delta p_{i,k}^{B,\text{left}} \right) \right), \quad (38)$$

$$z_i^{\text{go}} = \text{clip}_{[-500,500]} \left(\text{round} (100 \Delta h_{i,k}) \right) \quad (39)$$

として、

$$\text{go } x_i^{\text{go}} \ y_i^{\text{go}} \ z_i^{\text{go}} \ s_i^{\text{go}}$$

を送る。ただし、SDK 仕様上、go の各座標はおおむね $[-500, 500]$ cm、速度は 10–100 cm/s の範囲であり、三つの座標が同時に小さすぎる指令は受け付けられない [3]。したがって、 $|x_i^{\text{go}}|, |y_i^{\text{go}}|, |z_i^{\text{go}}|$ が同時に

小さい場合は go を送らず stop とする．本稿では実装を単純化する保守的な規則として， $\|\Delta \mathbf{p}_{i,k}^E\| < 0.2$ m なら go を送らない．なお，本稿では mission pad を使わない go を短距離 body-frame relative command として扱う．利用する SDK ライブラリが mission-pad 座標系または別の座標解釈を採用する場合は，式 (31) に対応する座標変換を実装側で変更する．また，go はコマンド完了待ちや応答遅れの影響を受けやすいので，複数機に対しては Python の thread/asyncio でほぼ同時に送信し，対象物速度は低速に制限する．

■**本稿での結論．** 協調取り囲みの本質は，対象物・スロット・近傍との相対位置から望ましい移動方向を計算し，安全シールドで補正し，各機へ送ることである．したがって，Tello に低レベル制御入力がないとしても，rc を用いた短周期フィードバック，または go を用いた低レート waypoint 追従に落とし込めば，協調取り囲みは実装可能である．論文では，主実装を rc モード，デバッグおよび安全な簡易実験を go/primitive モードとして整理するとよい．

4 協調取り囲み制御

4.1 正多角形による取り囲み目標

最も基本的な取り囲み目標は，対象物の周りに半径 R の正 N 角形を作ることである．対象物の進行方向角は，速度が小さいときと $\pm\pi$ の折り返しで不連続になりやすい．そこで，生の角度

$$\varphi_k^{\text{raw}} = \text{atan2}([\mathbf{v}_{o,k}]_2, [\mathbf{v}_{o,k}]_1) \quad (40)$$

を用意し， $\|\mathbf{v}_{o,k}\|_2 \geq v_{\text{eps}}$ のときだけ

$$\delta_k^\varphi = \text{atan2}(\sin(\varphi_k^{\text{raw}} - \varphi_{k-1}), \cos(\varphi_k^{\text{raw}} - \varphi_{k-1})), \quad (41)$$

$$\varphi_k = \varphi_{k-1} + \lambda_\varphi \delta_k^\varphi, \quad 0 < \lambda_\varphi \leq 1. \quad (42)$$

速度が小さい $\|\mathbf{v}_{o,k}\|_2 < v_{\text{eps}}$ のときは，角度を更新せず $\varphi_k = \varphi_{k-1}$ とする．このように角度差を $(-\pi, \pi]$ に折り返してからローパスフィルタをかけると， 179° から -179° へ変化した場合でも不要な大回転を避けられる． $v_{\text{eps}} > 0$ は速度がほぼゼロのときに角度が不安定になるのを避けるための小さい定数である．エージェント i の目標スロット角を

$$\alpha_{i,k}^{\text{reg}} = \varphi_k + \alpha_0 + \frac{2\pi(i-1)}{N} \quad (43)$$

とし，目標スロットを

$$\mathbf{r}_{i,k}^{\text{slot}} = \mathbf{r}_{o,k} + R \begin{bmatrix} \cos \alpha_{i,k}^{\text{reg}} \\ \sin \alpha_{i,k}^{\text{reg}} \end{bmatrix} \quad (44)$$

で与える． α_0 はスロット 1 を対象物のどの方向に置くかを定めるオフセットである．

4.2 対象物の進行方向に応じた配置変形

文献 [2] では，対象物が移動している場合に，対象物の進行方向を考慮して取り囲み配置を変形することも検討されている．本実験では，実装しやすい形として，速度が大きいほど進行方向側の角度間隔を小さくし，進行方向の反対側に大きな角度ギャップを置く設計を用いる．

速度の大きさを $\nu_k = \|\mathbf{v}_{o,k}\|_2$ と書く．後で CMDP の状態を s_k と書くため，速度スカラーには s_k を用いない． ν_k は単位を持つため，指数に直接入れず，代表速度 $v_{\text{scale}} > 0$ で無次元化する．

$$\nu_k = \|\mathbf{v}_{o,k}\|_2, \quad \bar{\nu}_k = \frac{\nu_k}{v_{\text{scale}}}, \quad \alpha_{\text{raw}}(\bar{\nu}_k) = \frac{2\pi}{N c_v \bar{\nu}_k}, \quad c_v > 1. \quad (45)$$

さらに，安全距離を保つために

$$\alpha_k = \max\{\alpha_{\min}, \alpha_{\text{raw}}(\bar{\nu}_k)\}, \quad \beta_k = 2\pi - (N-1)\alpha_k \quad (46)$$

と定義する． $\nu_k = 0$ のときに厳密に正多角形へ戻るためには， $\alpha_{\min} \leq 2\pi/N$ が必要である．また， $\beta_k \geq \alpha_k$ となり，反対側に大きなギャップが残るためにも同じ条件が自然である．したがって，隣接スロット間の最小距離が d_{drone} 以上となり，かつ停止時に正 N 角形へ戻るための実装上の可行性条件は

$$2 \arcsin\left(\frac{d_{\text{drone}}}{2R}\right) \leq \alpha_{\min} \leq \frac{2\pi}{N} \quad (47)$$

となる．この条件を満たす $\alpha_{\min}$ が存在するためには

$$R \geq \frac{d_{\text{drone}}}{2 \sin(\pi/N)} \quad (48)$$

が必要である． $d_{\text{drone}} > 2R$ の場合だけでなく，(48) を満たさない場合にも，指定した取り囲み半径で全隣接距離を安全距離以上に保つ正 N 角形配置は存在しない．

進行方向側の密な弧が φ_k の周りに来るように，角度列は次のように中心化して定義する．

$$\alpha_{i,k}^{\text{dir}} = \varphi_k - \frac{N-1}{2}\alpha_k + (i-1)\alpha_k, \quad i = 1, 2, \dots, N. \quad (49)$$

このとき，隣接角度差の多くは α_k ，最後に閉じる角度差は β_k となり，大きなギャップは進行方向の反対側に置かれる．目標スロットは

$$\mathbf{r}_{i,k}^{\text{slot}} = \mathbf{r}_{o,k} + R \begin{bmatrix} \cos \alpha_{i,k}^{\text{dir}} \\ \sin \alpha_{i,k}^{\text{dir}} \end{bmatrix} \quad (50)$$

で定義する．以後，Case 2 の進行方向依存配置を評価指標で明示するときは，式 (50) で定義されるスロットを

$$\mathbf{r}_{i,k}^{\text{slot,dir}} := \mathbf{r}_{i,k}^{\text{slot}} \quad (\text{Case 2}) \quad (51)$$

と書く．この配置は一般に正多角形ではないため，スロットの平均位置は対象物位置と一致しない．そこで

$$\bar{\mathbf{r}}_k^{\text{slot}} = \frac{1}{N} \sum_{i=1}^N \mathbf{r}_{i,k}^{\text{slot}}, \quad \mathbf{c}_k = \bar{\mathbf{r}}_k^{\text{slot}} - \mathbf{r}_{o,k} \quad (52)$$

を定義し，Case 2 の重心タスクでは $\mathbf{r}_{o,k}$ ではなく $\bar{\mathbf{r}}_k^{\text{slot}} = \mathbf{r}_{o,k} + \mathbf{c}_k$ を目標にする．これは，文献 [2] の Case 2 における「対象物の進行方向に応じて群重心目標を補正する」考え方に対応する．

■実装上の補足．式 (42) により，進行方向角は $\pm\pi$ の折り返しを考慮して更新する．折れ線軌道や停止・再始動の実験では， v_{eps} をやや大きくし，対象物速度が十分大きいときだけ φ_k とスロットを更新すると安全である．

4.3 逆運動学に基づく参照速度

強化学習だけで取り囲みを学習させると、初期学習が不安定になることがある。そこで、文献 [2] の逆運動学を「教師信号」または「ベースライン」として使う。最も単純な参照速度は、スロット追従速度

$$\mathbf{v}_{i,k}^{\text{slot}} = \dot{\mathbf{r}}_{o,k} + R\dot{\alpha}_{i,k} \begin{bmatrix} -\sin \alpha_{i,k} \\ \cos \alpha_{i,k} \end{bmatrix} + K_p (\mathbf{r}_{i,k}^{\text{slot}} - \mathbf{r}_{i,k}) \quad (53)$$

である。ここで $\alpha_{i,k}$ は Case 1 では $\alpha_{i,k}^{\text{reg}}$, Case 2 では $\alpha_{i,k}^{\text{dir}}$ を表す。 $\dot{\alpha}_{i,k}$ は角度差を $(-\pi, \pi]$ に折り返した有限差分で計算する。この式には対象物速度とスロット角速度が陽に入っているため、移動対象物、移動スロットに対するフィードフォワード速度として解釈できる。

文献 [2] に近い書き方をすると、タスク変数はドローン位置だけでなく、対象物位置、対象物速度、スロット位置、変形後辺長などの外生変数にも依存する。そこで外生変数をまとめて χ_k と書き、

$$\sigma_T = f_T(\mathbf{r}, \chi_k), \quad \mathbf{r} = [\mathbf{r}_1^\top, \mathbf{r}_2^\top, \dots, \mathbf{r}_N^\top]^\top, \quad \mathbf{v} = [\mathbf{v}_1^\top, \mathbf{v}_2^\top, \dots, \mathbf{v}_N^\top]^\top \in \mathbb{R}^{2N} \quad (54)$$

とする。このとき時間微分は、一般に

$$\dot{\sigma}_T = J_T(\mathbf{r}, \chi_k) \mathbf{v} + b_T(\mathbf{r}, \chi_k, \dot{\chi}_k) \quad (55)$$

となる。ここで J_T はドローン速度 $\mathbf{v}$ に関するヤコビ行列であり、 b_T は移動対象物、移動スロット、時間変化する辺長から生じるドリフト項である。目標タスク速度を実現する最小ノルム速度は

$$\mathbf{v}_T = J_{T,\lambda}^\dagger (\dot{\sigma}_{T,d} + \Lambda_T(\sigma_{T,d} - \sigma_T) - b_T) \quad (56)$$

で与える。数値実装では、特異姿勢や初期重なりに対する頑健性のため、Moore–Penrose 疑似逆行列ではなく damped pseudoinverse

$$J_{T,\lambda}^\dagger = J_T^\top (J_T J_T^\top + \lambda_T^2 I)^{-1}, \quad \lambda_T > 0 \quad (57)$$

を用いる。複数タスクを優先順位付きで統合する Null-Space-based Behavior (NSB) 型の速度は、低優先タスクのヤコビ行列を高優先タスクの零空間へ射影してから解く再帰形式で書くのが安全である。 L 個のタスクについて

$$\mathbf{u}_T = \dot{\sigma}_{T,d} + \Lambda_T(\sigma_{T,d} - \sigma_T) - b_T, \quad T = 1, \dots, L, \quad (58)$$

$$N_0 = I, \quad \mathbf{v}^{(0)} = 0, \quad (59)$$

$$\tilde{J}_T = J_T N_{T-1}, \quad (60)$$

$$\Delta \mathbf{v}_T = N_{T-1} \tilde{J}_{T,\lambda}^\dagger (\mathbf{u}_T - J_T \mathbf{v}^{(T-1)}), \quad (61)$$

$$\mathbf{v}^{(T)} = \mathbf{v}^{(T-1)} + \Delta \mathbf{v}_T, \quad (62)$$

$$N_T = N_{T-1} (I - \tilde{J}_{T,\lambda}^\dagger \tilde{J}_T), \quad (63)$$

と更新し、最終的に

$$\mathbf{v}^{\text{NSB}} = \mathbf{v}^{(L)} \quad (64)$$

とする。ここで $\tilde{J}_{T,\lambda}^\dagger = \tilde{J}_T^\top (\tilde{J}_T \tilde{J}_T^\top + \lambda_T^2 I)^{-1}$ である。ダンピングなしの Moore–Penrose 疑似逆行列を用いる理想的な場合には、 $I - \tilde{J}_T^\dagger \tilde{J}_T$ は零空間への射影として働くため、タスク T の補正 $\Delta \mathbf{v}_T$ は高

優先タスク $1, 2, \dots, T-1$ を乱さない方向に入る．一方，本稿の実装で用いる damped pseudoinverse では，特異姿勢近傍の数値安定性を優先するため，この零空間性は厳密ではなく近似的に満たされる．したがって，式 (61)–(63) は「数値安定化された優先順位付き NSB 近似」と解釈する．単純に $\mathbf{v}_1 + (I - J_1^\dagger J_1)\{\mathbf{v}_2 + (I - J_2^\dagger J_2)\mathbf{v}_3\}$ と重ねる簡略式を実装上のヒューリスティックとして使う場合は，厳密な優先順位付き NSB ではなく「簡略 NSB 型速度」と明記する．

実装に使うタスク，ヤコビ行列，ドリフト項を具体的に書く．正多角形 Case 1 では

$$\sigma_c^{\text{reg}} = \bar{\mathbf{r}} - \mathbf{r}_o, \quad \bar{\mathbf{r}} = \frac{1}{N} \sum_{i=1}^N \mathbf{r}_i, \quad (65)$$

$$\sigma_R = \begin{bmatrix} \frac{1}{2} \left(\|\mathbf{r}_1 - \mathbf{r}_o\|_2^2 - R^2 \right) \\ \vdots \\ \frac{1}{2} \left(\|\mathbf{r}_N - \mathbf{r}_o\|_2^2 - R^2 \right) \end{bmatrix}, \quad (66)$$

$$\sigma_e^{\text{reg}} = \begin{bmatrix} \frac{1}{2} \left(\|\mathbf{r}_1 - \mathbf{r}_2\|_2^2 - \ell_R^2 \right) \\ \vdots \\ \frac{1}{2} \left(\|\mathbf{r}_N - \mathbf{r}_1\|_2^2 - \ell_R^2 \right) \end{bmatrix}, \quad \ell_R = 2R \sin \frac{\pi}{N} \quad (67)$$

となる．さらに，エージェント番号と目標スロット番号の対応まで固定して E_{slot} で公平に比較する場合は，低優先のスロット誤差タスク

$$\sigma_o^{\text{reg}} = \begin{bmatrix} \mathbf{r}_1 - \mathbf{r}_1^{\text{slot}} \\ \vdots \\ \mathbf{r}_N - \mathbf{r}_N^{\text{slot}} \end{bmatrix} \quad (68)$$

も用いる．重心・半径・辺長だけでは正多角形全体の回転角が固定されず，どのエージェントがどのスロットを担当するかは一意に定まらないためである．Case 1 のドリフト項は

$$b_c^{\text{reg}} = -\mathbf{v}_o, \quad [b_R]_i = -(\mathbf{r}_i - \mathbf{r}_o)^\top \mathbf{v}_o, \quad b_e^{\text{reg}} = 0 \quad (69)$$

である．重心タスクや距離タスクでこれらを入れないと，移動対象物に対して $\mathbf{v}_o$ に相当するフィードフォワード補償が欠ける．

Case 2 では，式 (52) のスロット重心を用いて

$$\sigma_c^{\text{dir}} = \bar{\mathbf{r}} - \bar{\mathbf{r}}^{\text{slot}}, \quad \sigma_o = \begin{bmatrix} \mathbf{r}_1 - \mathbf{r}_1^{\text{slot}} \\ \vdots \\ \mathbf{r}_N - \mathbf{r}_N^{\text{slot}} \end{bmatrix} \quad (70)$$

とする．また，Case 2 の隣接距離タスクは一定長 ℓ_R ではなく，変形後スロットが要求する辺長

$$\ell_{i,k}^{\text{dir}} = \left\| \mathbf{r}_{i,k}^{\text{slot}} - \mathbf{r}_{i^+,k}^{\text{slot}} \right\|_2 \quad (71)$$

を用いて

$$\sigma_e^{\text{dir}} = \begin{bmatrix} \frac{1}{2} \left(\|\mathbf{r}_1 - \mathbf{r}_2\|_2^2 - (\ell_{1,k}^{\text{dir}})^2 \right) \\ \vdots \\ \frac{1}{2} \left(\|\mathbf{r}_N - \mathbf{r}_1\|_2^2 - (\ell_{N,k}^{\text{dir}})^2 \right) \end{bmatrix} \quad (72)$$

とする。Case 2 のドリフト項は

$$b_e^{\text{dir}} = -\dot{\mathbf{r}}^{\text{slot}}, \quad b_o = -\begin{bmatrix} \dot{\mathbf{r}}_1^{\text{slot}} \\ \vdots \\ \dot{\mathbf{r}}_N^{\text{slot}} \end{bmatrix}, \quad [b_e^{\text{dir}}]_i = -\ell_{i,k}^{\text{dir}} \dot{\ell}_{i,k}^{\text{dir}} \quad (73)$$

である。ここで $\dot{\mathbf{r}}_i^{\text{slot}}$, $\dot{\mathbf{r}}^{\text{slot}}$, $\dot{\ell}_{i,k}^{\text{dir}}$ は、式 (42) の角度折り返しを用いた有限差分、または対象物軌道が既知の場合の解析微分で計算する。

各タスクのヤコビ行列は次のように書ける。重心タスクでは

$$J_c = \frac{1}{N} \begin{bmatrix} I_2 & I_2 & \cdots & I_2 \end{bmatrix} \in \mathbb{R}^{2 \times 2N} \quad (74)$$

となる。また、距離タスク σ_R では、 $J_R \in \mathbb{R}^{N \times 2N}$ の i 行 j ブロックが

$$[J_R]_{ij} = \begin{cases} (\mathbf{r}_i - \mathbf{r}_o)^\top, & j = i \\ 0_{1 \times 2}, & j \neq i \end{cases} \quad (75)$$

となる。さらに、隣接距離タスクでは、 i^+ はフォーメーショングラフ $\mathcal{G}_{\text{form}}$ 上の次ノードとし、サイクルの場合は $N^+ = 1$ として

$$[J_e]_{ij} = \begin{cases} (\mathbf{r}_i - \mathbf{r}_{i^+})^\top, & j = i \\ -(\mathbf{r}_i - \mathbf{r}_{i^+})^\top, & j = i^+ \\ 0_{1 \times 2}, & \text{otherwise} \end{cases} \quad (76)$$

となる。スロット誤差タスク σ_o および σ_o^{reg} では $J_o = I_{2N}$, $b_o = -[\dot{\mathbf{r}}_1^{\text{slot}^\top}, \dot{\mathbf{r}}_2^{\text{slot}^\top}, \dots, \dot{\mathbf{r}}_N^{\text{slot}^\top}]^\top$ である。したがって、Python 実装では、各タスクについて $e_T = \sigma_{T,d} - \sigma_T$ と $\mathbf{u}_T = \dot{\sigma}_{T,d} + \Lambda_T e_T - b_T$ を計算し、式 (59)–(64) で優先順位に従って合成すればよい。

本稿の Case 1 は、Tello 実装で扱いやすいように対象物位置 $\mathbf{r}_o$ を直接基準とする距離タスク σ_R を用いる。文献 [2] の Case 1 では、群平均位置から各ビークルまでの距離を用いる記述も含まれるため、厳密に原論文のタスクを再現したい場合は、 σ_R を $\frac{1}{2}(\|\mathbf{r}_i - \bar{\mathbf{r}}\|^2 - R^2)$ 型に置き換える。ただし、 $\sigma_c^{\text{reg}} = 0$ が十分小さくなれば $\bar{\mathbf{r}} \simeq \mathbf{r}_o$ であり、本稿の簡略化は実装上自然である。

Case 1 では、形状フォーメーションのみを比較する場合は

$$(\sigma_1, \sigma_2, \sigma_3) = (\sigma_e^{\text{reg}}, \sigma_c^{\text{reg}}, \sigma_R) \quad (77)$$

とする。一方、本稿の評価指標 E_{slot} のようにエージェント i とスロット i の対応も評価する場合は、NSB 比較手法にも低優先のスロット追従タスクを加え、

$$(\sigma_1, \sigma_2, \sigma_3, \sigma_4) = (\sigma_e^{\text{reg}}, \sigma_c^{\text{reg}}, \sigma_R, \sigma_o^{\text{reg}}) \quad (78)$$

として実装する。Case 2 では、進行方向依存の辺長とスロット重心を用いて

$$(\sigma_1, \sigma_2, \sigma_3) = (\sigma_e^{\text{dir}}, \sigma_c^{\text{dir}}, \sigma_o) \quad (79)$$

とする。

■本研究での使い分け. NSB 速度 (64) は, (i) 比較手法, (ii) 強化学習の初期教師, (iii) 方策入力の一部, (iv) コスト関数中の参照速度, のいずれにも使える. SMC Otani 型アルゴリズムの主役は方策学習なので, 最初は $\mathbf{v}_i^{\text{NSB}}$ または $\mathbf{v}_i^{\text{slot}}$ をコストに入れ, 十分に安定した後でその重みを小さくする設計が扱いやすい. 本稿の NSB 比較では辺長タスクを高優先に置くが, これはフォーメーション形状を崩さないための設計であり, 衝突回避・壁回避・高度制限は NSB ではなく安全制約コストおよび Safety Supervisor が担当する. この役割分担により, 取り囲み形状の性能評価と実機安全監視を混同しない.

5 CMDP と制約付き方策探索としての定式化

5.1 状態, 観測, 行動

安全シールドを用いる場合の記法を先に定める. 方策が確率的に出す行動を raw action $\tilde{\mathbf{a}}_{i,k}$ とし, 実際に Unity 環境または Tello SDK コマンド層へ入力される行動を executed action $\mathbf{a}_{i,k}$ とする. 安全シールドがない数値実験では $\mathbf{a}_{i,k} = \tilde{\mathbf{a}}_{i,k}$ であり, 安全シールドを使う場合は $\mathbf{a}_{i,k} = S_i(s_k, \tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k})$ と書く. 目的コストと制約コストは, 特に断らない限り, 閉ループ環境に実際に入力された $\mathbf{a}_{i,k}$ に対して評価する. 一方, 方策勾配の log-probability は, 方策がサンプルした raw action $\tilde{\mathbf{a}}_{i,k}$ に対して計算する.

全体状態は, 目的コストの入力変化項 $\|\mathbf{a}_{i,k} - \mathbf{a}_{i,k-1}\|^2$, 式 (42) のローパス済み進行方向角 φ_k , および対象物軌道の切替時刻・モードを一意に決められるよう, 前回入力とスロット生成に必要な内部状態を含めて

$$s_k = \left(\{\mathbf{r}_{i,k}, \mathbf{v}_{i,k}, h_{i,k}, \dot{h}_{i,k}, \psi_{i,k}, \dot{\psi}_{i,k}, \mathbf{a}_{i,k-1}\}_{i=1}^N, \mathbf{r}_{o,k}, \mathbf{v}_{o,k}, \varphi_k, \{\mathbf{r}_{i,k}^{\text{slot}}\}_{i=1}^N, \zeta_k^o, \mathcal{O}_{\text{obs}}, \mathcal{B}_k \right) \quad (80)$$

とする. ζ_k^o は対象物軌道の内部状態であり, 円軌道の位相, 折れ線軌道の区間番号, 境界反射フラグ, 目標速度の切替タイマなどを含む. この項を入れることで, 時間依存またはモード依存の対象物運動でも, s_k から s_{k+1} の分布が定まる Markov 的な記述になる. $\mathcal{O}_{\text{obs}}$ は障害物集合, $\mathcal{B}_k$ は各 Tello のバッテリーや通信状態などの実験状態を表す. 数値実験だけなら $\mathcal{B}_k$ は省略してよい. $\{\mathbf{r}_{i,k}^{\text{slot}}\}$ は $\mathbf{r}_{o,k}, \mathbf{v}_{o,k}, \varphi_k, \zeta_k^o$ から決定されるため冗長に見えるが, CMDP の状態としては, 低域通過フィルタ後のスロットが環境内部状態であることを明示するために含める. 初期時刻では $\mathbf{a}_{i,-1} = 0$ とし, hover 相当の executed action を前回入力として扱う. raw action は方策勾配用にロールアウトバッファへ保存するが, CMDP の遷移とコストには安全シールド後の executed action を用いる.

エージェント i の局所観測は固定長ベクトルとして構成する. MLP 方策を使う場合, 時刻によって $|\mathcal{N}_i^{\text{obs}}|$ が変わると入力次元が変動するため, 最大観測近傍数 $n_{\text{obs}}^{\text{max}}$ を決め, 観測可能な近傍を距離の近い順に並べる. ℓ 番目の観測近傍を $j_{i,\ell,k}$ とし, 近傍が存在しないスロットにはゼロを入れ, $\text{mask}_{i,\ell,k} \in \{0, 1\}$ を用いる. また, 長方形飛行領域の水平面射影を $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ とし, 壁方向の余裕ベクトルを

$$\mathbf{d}_{i,k}^{\text{wall,vec}} = [x_{i,k} - x_{\min} \quad x_{\max} - x_{i,k} \quad y_{i,k} - y_{\min} \quad y_{\max} - y_{i,k}]^T \quad (81)$$

で定義する．このとき，例えば

$$o_{i,k} = \left[\frac{\mathbf{r}_{i,k} - \mathbf{r}_{o,k}}{R_{\max}}, \frac{\mathbf{r}_{i,k} - \mathbf{r}_{i,k}^{\text{slot}}}{R_{\max}}, \frac{\mathbf{v}_{i,k}}{v_{\max}}, \frac{\mathbf{v}_{o,k}}{v_{o,\max}}, \sin \psi_{i,k}, \cos \psi_{i,k}, \frac{\dot{\psi}_{i,k}}{\omega_{\psi}^{\max}}, \frac{h_{i,k} - h^{\text{ref}}}{h_{\text{scale}}}, \frac{\dot{h}_{i,k}}{v_{z,\max}}, \left\{ m_{i,\ell,k} \frac{\mathbf{r}_{j_{i,\ell,k}} - \mathbf{r}_{i,k}}{R_{\max}}, m_{i,\ell,k} \frac{\mathbf{v}_{j_{i,\ell,k}} - \mathbf{v}_{i,k}}{v_{\max}}, m_{i,\ell,k} \right\}_{\ell=1}^{n_{\text{obs}}^{\max}}, \frac{\mathbf{d}_{i,k}^{\text{wall,vec}}}{d_{\max}^{\text{wall}}}, \frac{b_{i,k}}{100}, \mathbf{a}_{i,k-1}, \sin \frac{2\pi(i-1)}{N}, \cos \frac{2\pi(i-1)}{N} \right] \quad (82)$$

とする．ここで $\mathcal{N}_i^{\text{obs}}$ は局所観測に含める候補近傍であり，通信近傍 $\mathcal{N}_i^{\text{comm}}$ と同じにしてもよいし，外部計測系から得られる安全監視用近傍として別に定義してもよい．壁に関する特徴量は，最短壁距離だけでなく x 軸正負方向と y 軸正負方向の余裕を入れる．制約コストで用いる最短壁距離は $d_{i,k}^{\text{wall,min}} := \min_{\ell} [\mathbf{d}_{i,k}^{\text{wall,vec}}]_{\ell}$ と定義する．これにより，方策は「壁が近い」だけでなく「どちら側の壁が近い」も識別できる．最後の $\sin, \cos$ は，パラメータ共有方策であっても各ドローンの役割番号を識別できるように入れる特徴量である．可変個数の近傍を明示的に扱いたい場合は，この固定長 padding の代わりに， $\sum_{j \in \mathcal{N}_i^{\text{obs}}} \rho_{\theta}(\mathbf{r}_{j,k} - \mathbf{r}_{i,k}, \mathbf{v}_{j,k} - \mathbf{v}_{i,k})$ のような permutation-invariant 集約や GNN を使う．

本稿では，実装と論文上の分散性を切り分ける．**方策実行時の入力**は局所観測 $o_{i,k}$ である．一方，Unity 数値実験や実機初期検証では，訓練時の目的コスト・制約コスト・安全シールドは Unity 環境または外部計測 PC が全状態から計算し，各 Agent Process に返す centralized training/decentralized execution (CTDE) 型の評価器として扱う．したがって，学習更新における方策・乗数の通信は近傍通信に限定されるが，環境評価と安全監視は必ずしも近傍通信だけで実現されるわけではない．完全分散評価を強調したい場合は，後述の全ペア距離や重心項を近傍平均または consensus 推定値で置き換える．

行動は，物理的な推力や姿勢角ではなく，Tello SDK コマンドへ変換可能な抽象入力として定義する．主実装では rc a b c d を用いるため，方策がサンプルする連続型の正規化 raw action を

$$\tilde{\mathbf{a}}_{i,k} \in [-1, 1]^4, \quad \tilde{\mathbf{a}}_{i,k} = [\tilde{a}_{i,k}^{\text{lr}}, \tilde{a}_{i,k}^{\text{fb}}, \tilde{a}_{i,k}^{\text{ud}}, \tilde{a}_{i,k}^{\text{yaw}}]^{\top} \quad (83)$$

とする．方策は

$$\tilde{\mathbf{a}}_{i,k} \sim \pi_{\vartheta_i}(\cdot \mid o_{i,k}) \quad (84)$$

で与える．安全シールド後の executed action は

$$\mathbf{a}_{i,k} = S_i(s_k, \tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k}) \in [-1, 1]^4, \quad \mathbf{a}_{i,k} = [a_{i,k}^{\text{lr}}, a_{i,k}^{\text{fb}}, a_{i,k}^{\text{ud}}, a_{i,k}^{\text{yaw}}]^{\top} \quad (85)$$

であり，安全シールドを使わないときは S_i を恒等写像として $\mathbf{a}_{i,k} = \tilde{\mathbf{a}}_{i,k}$ とする．実機へは，executed action $\mathbf{a}_{i,k}$ を式 (35) により整数 SDK コマンド

$$\mathbf{u}_{i,k}^{\text{rc}} = [u_{i,k}^{\text{rc,lr}}, u_{i,k}^{\text{rc,fb}}, u_{i,k}^{\text{rc,ud}}, u_{i,k}^{\text{rc,yaw}}]^{\top} \in \{-100, \dots, 100\}^4 \quad (86)$$

へ変換して送る．したがって，方策は連続値を出すか，実機へ送られるのは Tello SDK で許される rc テキストコマンドだけである．より保守的な実機検証では，executed action または参照速度を直接 rc にせず，式 (36)–(39) で短距離 waypoint に変換し，go と stop だけを送る実装も可能である．実装上は，平均 $m_{\vartheta_i}(o)$ と標準偏差 $\sigma_{\vartheta_i}(o)$ を出力する tanh-squashed Gaussian 方策を用いるとよい．

5.2 目的コスト

SMC Otani 論文 [1] では目的を最小化問題として書いているため、本稿でも報酬ではなくコスト ℓ_i を用いる。以下の目的コストには群重心 $\bar{\mathbf{r}}$ やフォーメーショングラフ上の辺長誤差が含まれる。これらは、訓練時には Unity 環境または外部計測系が全状態から計算して各 Agent Process に返す。方策そのものは局所観測 $o_{i,k}$ のみを入力として実行されるため、centralized training/decentralized execution (CTDE) 型の評価として解釈する。重心参照点を

$$\mathbf{r}_k^{\text{cg,ref}} = \begin{cases} \mathbf{r}_{o,k}, & \text{Case 1} \\ \bar{\mathbf{r}}_k^{\text{slot}}, & \text{Case 2} \end{cases} \quad (87)$$

と定義する。Case 2 で $\bar{\mathbf{r}}_k^{\text{slot}}$ を使うのは、進行方向依存スロットの重心が一般に対象物位置 $\mathbf{r}_{o,k}$ と一致しないためである。また、フォーメーショングラフ上の隣接エージェント i, j の目標辺長を

$$\ell_{ij,k}^{\text{edge,ref}} = \|\mathbf{r}_{i,k}^{\text{slot}} - \mathbf{r}_{j,k}^{\text{slot}}\|_2 \quad (88)$$

とする。Case 1 では $\ell_{ij,k}^{\text{edge,ref}} = \ell_R$ であり、Case 2 では進行方向依存スロットに合わせて辺長が変化する。

式 (92) に現れる $\mathbf{v}_{i,k}^{xy,\text{cmd}} \in \mathbb{R}^2$ は、4 次元 executed action そのものではなく、式 (18) で定義される機体座標系の水平速度指令を、式 (20) により世界座標系へ変換した 2 次元速度である。前後・左右の最大速度が異なる場合は、世界座標の速度誤差を機体座標へ戻してから $D_v = \text{diag}(v_{\text{fb}}^{\text{max}}, v_{\text{lr}}^{\text{max}})$ により正規化する。すなわち

$$R_{BE}(\psi_{i,k}) = \begin{bmatrix} \cos \psi_{i,k} & \sin \psi_{i,k} \\ -\sin \psi_{i,k} & \cos \psi_{i,k} \end{bmatrix}, \quad \left\| D_v^{-1} R_{BE}(\psi_{i,k}) (\mathbf{v}_{i,k}^{xy,\text{cmd}} - \mathbf{v}_{i,k}^{\text{ref,clip}}) \right\|_2^2 \quad (89)$$

を用いる。これは D_v が世界座標軸ではなく Tello の前後・左右軸に対応するためである。yaw 参照は実験目的に応じて

$$\psi_{i,k}^{\text{ref}} = \begin{cases} \varphi_k, & \text{対象物の進行方向へ機体正面を向ける場合} \\ \text{atan2}([\mathbf{r}_{o,k} - \mathbf{r}_{i,k}]_2, [\mathbf{r}_{o,k} - \mathbf{r}_{i,k}]_1), & \text{対象物をカメラで注視する場合} \end{cases} \quad (90)$$

として定義する。参照速度 $\mathbf{v}_{i,k}^{\text{ref}}$ が実機または Unity の最大水平速度を超えると、実現不可能な参照への追従誤差が過大になり、学習が不安定になることがある。そこで、目的コストでは

$$\mathbf{v}_{i,k}^{\text{ref,clip}} = \min \left\{ 1, \frac{v_{\text{max}}}{\max\{\|\mathbf{v}_{i,k}^{\text{ref}}\|_2, \epsilon_v\}} \right\} \mathbf{v}_{i,k}^{\text{ref}}, \quad \epsilon_v > 0 \quad (91)$$

を用いる。

エージェント i の 1 ステップ目的コストを

$$\begin{aligned}
\ell_i(s_k, \mathbf{a}_k, s_{k+1}) = & w_{\text{slot}} \frac{\|\mathbf{r}_{i,k+1} - \mathbf{r}_{i,k+1}^{\text{slot}}\|_2^2}{R^2} + w_R \left(\frac{\|\mathbf{r}_{i,k+1} - \mathbf{r}_{o,k+1}\|_2 - R}{R} \right)^2 \\
& + w_c \frac{\|\bar{\mathbf{r}}_{k+1} - \mathbf{r}_{k+1}^{\text{cg,ref}}\|_2^2}{R^2} + w_e \sum_{j \in \{i^-, i^+\}} \left(\frac{\|\mathbf{r}_{i,k+1} - \mathbf{r}_{j,k+1}\|_2 - \ell_{ij,k+1}^{\text{edge,ref}}}{\ell_{ij,k+1}^{\text{edge,ref}}} \right)^2 \\
& + w_v \frac{\|\mathbf{v}_{i,k}^{\text{xy,cmd}} - \mathbf{v}_{i,k}^{\text{ref,clip}}\|_2^2}{(v_{\text{max}})^2} + w_u \|\mathbf{a}_{i,k}\|_2^2 + w_{\Delta u} \|\mathbf{a}_{i,k} - \mathbf{a}_{i,k-1}\|_2^2 \\
& + w_h \left(\frac{h_{i,k+1} - h^{\text{ref}}}{h_{\text{scale}}} \right)^2 + w_\psi (1 - \cos(\psi_{i,k+1} - \psi_{i,k+1}^{\text{ref}})) \quad (92)
\end{aligned}$$

とする。ここで、 i^- , i^+ はフォーメーショングラフ $\mathcal{G}_{\text{form}}$ 上の前後隣接エージェントである。参照速度 $\mathbf{v}_{i,k}^{\text{ref}}$ は、式 (53) の $\mathbf{v}_i^{\text{slot}}$ または式 (64) の $\mathbf{v}_i^{\text{NSB}}$ から取り出したエージェント i の水平速度であり、コスト中では式 (91) の飽和済み参照 $\mathbf{v}_{i,k}^{\text{ref,clip}}$ を使う。 $k=0$ では入力変化項の初期値を $\mathbf{a}_{i,-1} = 0$, すなわち hover 相当入力とする。水平速度正規化の v_{max} は、 $v_{\text{fb}}^{\text{max}} = v_{\text{lr}}^{\text{max}}$ のとき $v_{\text{max}} = v_{\text{fb}}^{\text{max}}$ と置く。前後・左右の最大速度を変える場合は、 $\|\mathbf{v}_{i,k}^{\text{xy,cmd}} - \mathbf{v}_{i,k}^{\text{ref,clip}}\|_2^2 / (v_{\text{max}})^2$ の代わりに、式 (89) のように世界座標の速度誤差を機体座標へ戻してから正規化する。また、 $\sum_{j \in \{i^-, i^+\}}$ を全エージェントで合計すると、無向サイクル上の各辺は両端から二重に数えられる。本稿ではこの効果を含めて w_e を調整する。二重カウントを避けたい場合は、各 i について $j = i^+$ のみを足すか、辺長項に係数 1/2 を掛ける。

典型的な初期重みは

$$w_{\text{slot}} = 5.0, \quad w_R = 1.0, \quad w_c = 1.0, \quad w_e = 1.0, \quad (93)$$

$$w_v = 0.2, \quad w_u = 10^{-3}, \quad w_{\Delta u} = 10^{-2}, \quad w_h = 3.0, \quad w_\psi = 0.1 \quad (94)$$

である。まず w_v を大きめに逆運動学参照速度に近い行動を学習させ、徐々に w_v を小さくすると、方策が自分で取り囲み戦略を改善しやすくなる。

5.3 安全制約コスト

制約コスト $C_i(s_k, \mathbf{a}_k, s_{k+1}) \in \mathbb{R}^{m_i}$ は、次のように設定する。

$$C_{i,1}(s_k, \mathbf{a}_k, s_{k+1}) = \left[1 - \frac{\min_{j \neq i} \|\mathbf{r}_{i,k+1} - \mathbf{r}_{j,k+1}\|_2}{d_{\text{drone}}} \right]_+, \quad (95)$$

$$C_{i,2}(s_k, \mathbf{a}_k, s_{k+1}) = \left[1 - \frac{\|\mathbf{r}_{i,k+1} - \mathbf{r}_{o,k+1}\|_2}{d_{\text{target}}} \right]_+, \quad (96)$$

$$C_{i,3}(s_k, \mathbf{a}_k, s_{k+1}) = \left[1 - \frac{d_{i,k+1}^{\text{wall,min}}}{d_{\text{wall}}^{\text{safe}}} \right]_+, \quad (97)$$

$$C_{i,4}(s_k, \mathbf{a}_k, s_{k+1}) = \left[\frac{|h_{i,k+1} - h^{\text{ref}}| - e_h}{e_h} \right]_+, \quad (98)$$

$$C_{i,5}(s_k, \mathbf{a}_k, s_{k+1}) = \left[\frac{\|\mathbf{a}_{i,k}\|_\infty - a_{\text{safe}}}{1 - a_{\text{safe}}} \right]_+. \quad (99)$$

ここで $[z]_+ = \max\{z, 0\}$ である。 $\mathbf{a}_{i,k}$ は executed action であるため、安全シールドを使う実験では式 (99) も実際に送られた入力の飽和度を測る。raw policy 自体の危険傾向を評価したい場合は、同じ式

を $\tilde{\mathbf{a}}_{i,k}$ に適用した C_i^{raw} を別途ログに残す. d_{drone} は機体間安全距離, d_{target} は対象物との最小距離, $d_{\text{wall}}^{\text{safe}}$ は壁からの安全距離, e_h は許容高度誤差, $a_{\text{safe}} < 1$ は実機での安全な入力範囲を表す. $C_{i,1}$ は全ペア最小距離を用いるため, Unity や実機初期検証では centralized safety evaluator が全状態から計算する. 完全分散評価に寄せる場合は,

$$C_{i,1}^{\text{local}}(s_k, \mathbf{a}_k, s_{k+1}) = \left[1 - \frac{\min_{j \in \mathcal{N}_i^{\text{safe}}} \|\mathbf{r}_{i,k+1} - \mathbf{r}_{j,k+1}\|_2}{d_{\text{drone}}} \right]_+ \quad (100)$$

のように, 安全グラフ $\mathcal{G}_{\text{safe}}$ 上の近傍だけで評価する.

制約値のスケールをしきい値 d_i と合わせるため, 本稿では割引和そのものではなく, 正規化済み割引平均を用いる.

$$\kappa_{\gamma,H} = \begin{cases} \frac{1-\gamma}{1-\gamma^H}, & 0 < \gamma < 1 \\ \frac{1}{H}, & \gamma = 1 \end{cases} \quad (101)$$

とし, 制約関数を

$$G_i(\vartheta) = \mathbb{E} \left[\kappa_{\gamma,H} \sum_{k=0}^{H-1} \gamma^k C_i(s_k, \mathbf{a}_k, s_{k+1}) \right] \quad (102)$$

で定義する. この定義なら, C_i の各成分がおおむね $[0, 1]$ の範囲にあるとき, G_i も平均違反率に近いスケールになり, $d_i = 0.01$ や 0.02 の値が自然に解釈できる. もし非正規化の割引和 $\sum_k \gamma^k C_i$ を用いる場合は, しきい値も $d_i / \kappa_{\gamma,H}$ 程度へ拡大する必要がある.

初期値としては

$$d_{\text{drone}} = 0.50 \text{ m}, \quad d_{\text{target}} = 0.35 \text{ m}, \quad d_{\text{wall}}^{\text{safe}} = 0.35 \text{ m}, \quad e_h = 0.20 \text{ m}, \quad a_{\text{safe}} = 0.8, \quad (103)$$

および

$$d_i = d = (0.02, 0.01, 0.02, 0.02, 0.02)^\top \quad (104)$$

程度から始める. ここでは全エージェントで $m_i = 5$ とし, 同じしきい値ベクトル $d_i = d \in \mathbb{R}^5$ を用いる. エージェントごとに制約成分数やしきい値を変える場合は, $d_i \in \mathbb{R}^{m_i}$ として $G_i(\vartheta) \leq d_i$ を成分ごとに評価する. 実機では, これらの制約コストは「学習上の平均的な安全指標」であり, 瞬間的な衝突・壁接近・高度逸脱の抑制は後述の安全シールドで別途扱う. 安全シールドも, 計測遅延, 速度飽和, QP の可解性, 非常停止条件が満たされる範囲で機能する工学的な防護層であり, 学習上の平均制約と混同しない.

5.4 制約付き方策探索問題

各エージェントの目的関数と正規化制約関数を

$$F_i(\vartheta) = \mathbb{E}_{\tau \sim p(\cdot | \pi^\vartheta, S)} \left[\sum_{k=0}^{H-1} \gamma^k \ell_i(s_k, \mathbf{a}_k, s_{k+1}) \right] \quad (105)$$

$$G_i(\vartheta) = \mathbb{E}_{\tau \sim p(\cdot | \pi^\vartheta, S)} \left[\kappa_{\gamma,H} \sum_{k=0}^{H-1} \gamma^k C_i(s_k, \mathbf{a}_k, s_{k+1}) \right] \quad (106)$$

とする．ここで $p(\cdot | \pi^\vartheta, S)$ は、raw action を方策 π^ϑ からサンプルし、必要に応じて安全シールド S を通した executed action $\mathbf{a}_k$ で環境を遷移させる閉ループ軌道分布を表す． ϑ は共有方策パラメータを表す．目的関数 F_i は学習性能を表す非正規化割引コストとして扱う一方、制約関数 G_i はしきい値 d_i を平均違反率として解釈しやすくするために正規化済み割引平均として定義する．このため Lagrangian 内では目的項と制約項の数値スケールが異なるので、 $\eta_{\mu,q}$, $\mu_{\max}$, およびコスト重みは、正規化済み制約値 G_i のスケールに合わせて調整する．実際の分散実装では、各エージェント i がその局所コピー ϑ_i をもつ．全体の制約付き方策探索問題は

$$\min_{\vartheta \in X} \frac{1}{N} \sum_{i=1}^N F_i(\vartheta) \quad (107)$$

$$\text{subject to } G_i(\vartheta) \leq d_i, \quad i = 1, 2, \dots, N. \quad (108)$$

ここで、不等式 $G_i(\vartheta) \leq d_i$ は成分ごとの不等式として解釈する． X は方策パラメータの射影集合である．数値実験では、例えば

$$X = \{\vartheta \in \mathbb{R}^{n_\vartheta} \mid \|\vartheta\|_2 \leq R_X\} \quad \Pi_X(z) = \begin{cases} z, & \|z\|_2 \leq R_X, \\ R_X z / \|z\|_2, & \text{otherwise} \end{cases} \quad (109)$$

のような大きな Euclidean ball としておけば通常は射影が発動しない．PPO 実装で明示的な射影を入れない場合は、gradient clipping と weight decay による近似的な有界化として扱い、理論上の射影付き更新とは区別して記述する．

理想問題 (107)–(108) では全エージェントが共通パラメータ ϑ を共有する．一方、分散実装中は各エージェントが局所コピー $\vartheta_{i,q}$ を持つため、学習イテレーション q のロールアウトは

$$\pi_q(\tilde{\mathbf{a}}_k \mid s_k) = \prod_{i=1}^N \pi_{\vartheta_{i,q}}(\tilde{\mathbf{a}}_{i,k} \mid o_{i,k}) \quad (110)$$

という raw action に関する heterogeneous product policy に基づいて収集し、環境遷移とコスト評価には $\mathbf{a}_{i,k} = S_i(s_k, \tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k})$ を用いる．コンセンサス更新は、これらの局所コピーを共通解へ近づける役割を持つ．ただし、後述の式 (127) は、各エージェントの行動確率に関する independent actor-critic 型の局所勾配方向であり、共通パラメータ問題 (107)–(108) の厳密な方策勾配の不偏推定量ではない．厳密な共有パラメータ勾配を書く場合は、有限ホライズンの cost-to-go

$$\mathcal{R}_{i,k}^F = \sum_{\ell=k}^{H-1} \gamma^{\ell-k} \ell_i(s_\ell, \mathbf{a}_\ell, s_{\ell+1}) \quad (111)$$

を用いて、たとえば

$$\nabla_{\vartheta} F_i(\vartheta) = \mathbb{E} \left[\sum_{k=0}^{H-1} \gamma^k \left(\sum_{j=1}^N \nabla_{\vartheta} \log \pi_{\vartheta}(\tilde{\mathbf{a}}_{j,k} \mid o_{j,k}) \right) \mathcal{R}_{i,k}^F \right] \quad (112)$$

のように、他エージェント j の score function も含まれる．外側の γ^k は、 $\mathcal{R}_{i,k}^F$ を時刻 k から見た相対割引 cost-to-go として定義したために必要である．制約付き Lagrangian の勾配では、

$$\mathcal{R}_{i,k}^{\mathcal{L}}(y_i) = \sum_{\ell=k}^{H-1} \gamma^{\ell-k} \left\{ \frac{1}{N} \ell_i(s_\ell, \mathbf{a}_\ell, s_{\ell+1}) + \kappa_{\gamma, H} y_i^\top C_i(s_\ell, \mathbf{a}_\ell, s_{\ell+1}) \right\} \quad (113)$$

を用いて、式 (112) の $\mathcal{R}_{i,k}^F$ を $\mathcal{R}_{i,k}^{\mathcal{L}}(y_i)$ に置き換える．すなわち、Lagrangian の共有パラメータ勾配は

$$\nabla_{\vartheta} f_i(\vartheta, y) = \mathbb{E} \left[\sum_{k=0}^{H-1} \gamma^k \left(\sum_{j=1}^N \nabla_{\vartheta} \log \pi_{\vartheta}(\tilde{a}_{j,k} \mid o_{j,k}) \right) \mathcal{R}_{i,k}^{\mathcal{L}}(y_i) \right] \quad (114)$$

と表せる．したがって、本稿の実装版は Otani 論文の exact-gradient 理論を直接主張するものではなく、事象駆動型 consensus primal-dual 構造を用いた分散安全 RL surrogate として扱う．必要に応じて、中央評価器が return や advantage を計算して各エージェントに返す CTDE 型評価として実装できる [9]．

Otani 論文 [1] の理論設定は、有限状態 CMDP における割引無限和と exact gradient を基本にしている．本稿の Tello/Unity 設定は、連続状態・有限ホライズン・ロールアウト勾配を用いる実装版であるため、(107)–(108) は Otani 論文の理論問題そのものではなく、そのアルゴリズム構造を Tello 取り囲み実験へ移植するための有限ホライズン近似である．

制約乗数を $y = [y_1^{\top}, y_2^{\top}, \dots, y_N^{\top}]^{\top}$ とし、

$$M_i = [0, \mu_{\max}]^{m_i} \subset \mathbb{R}_+^{m_i}, \quad M = M_1 \times M_2 \times \dots \times M_N \quad (115)$$

とする．射影 Π_{M_i} は成分ごとの clipping

$$[\Pi_{M_i}(z)]_{\ell} = \min\{\mu_{\max}, \max\{0, z_{\ell}\}\} \quad (116)$$

である．compact-dual surrogate は

$$f(\vartheta, y) = \sum_{i=1}^N f_i(\vartheta, y), \quad f_i(\vartheta, y) = \frac{1}{N} F_i(\vartheta) + y_i^{\top} (G_i(\vartheta) - d_i), \quad (117)$$

であり、

$$\min_{\vartheta \in X} \max_{y \in M} f(\vartheta, y) \quad (118)$$

を分散的に解く．この compact-dual surrogate は、Otani 論文 [1] の事象駆動型 consensus primal-dual 更新の構造に対応する．ただし、本稿では exact gradient を有限ホライズン・ロールアウト推定に置き換えるため、理論保証をそのまま主張するのではなく、通信量削減と empirical constraint cost の低下を実験的に検証する．

6 各エージェントが持つ変数と交換する情報

エージェント i が持つ変数を表 4 にまとめる．

表 4: エージェント i が保持する主な変数

分類	記号	意味
状態推定	$\mathbf{r}_{i,k}, \mathbf{v}_{i,k}, h_{i,k}, \dot{h}_{i,k}$ $\psi_{i,k}, \dot{\psi}_{i,k}$	自機の位置, 速度, 高度, 上下速度, yaw 角, yaw 角速度. Unity では真値, 実機では外部カメラまたは motion capture から推定.
対象物推定	$\mathbf{r}_{o,k}, \mathbf{v}_{o,k}$	対象物の位置・速度. 外部カメラ, マーカ, または Unity から取得.
スロット内部状態	$\varphi_k, \mathbf{r}_{i,k}^{\text{slot}}$	ローパス済み進行方向角と各ドローンの目標スロット. Markov 性を保つため環境内部状態として保持する.
前回入力	$\mathbf{a}_{i,k-1}$	入力変化ペナルティと滑らかな制御のために保持する executed action. 初期値は hover 相当の 0 とする.
局所観測	$\mathbf{o}_{i,k}$	方策ネットワークの入力. 自機情報, 対象物相対位置, スロット誤差, 固定長化した近傍相対位置, 4 方向の壁距離, 前回入力などを連結したベクトル.
方策パラメータ	$\vartheta_{i,q}$	エージェント i が持つ共有方策パラメータの局所コピー.
価値関数パラメータ	$\xi_{i,q}$	advantage 推定用の価値関数パラメータ. 通信しないローカル変数として扱う.
乗数スタック	$\mu_{i,q}$	全制約乗数 $[\mu_{i,q}^{(1)}, \mu_{i,q}^{(2)}, \dots, \mu_{i,q}^{(N)}]$ の局所コピー. 自分の制約更新は $\mu_{i,q}^{(i)}$ のブロックに入れる.
送信済み値	$\hat{\vartheta}_{i,q}, \hat{\mu}_{i,q}$	最後に近傍へ送信した方策・乗数. イベントが発生しない限り更新しない.
近傍からの保存値	$\hat{\vartheta}_{j,q}, \hat{\mu}_{j,q}, j \in \mathcal{N}_i^{\text{comm}}$	通信近傍が最後に送ってきた値. コンセンサス混合に使う.
ロールアウト記録	$\{o, \tilde{a}, a, \ell, C, \log \pi\}$	raw action, executed action, コスト, 制約値, raw action の log-probability を含む勾配推定用データ.

通信で送るのは, 学習更新用の $\hat{\vartheta}_i$ と $\hat{\mu}_i$ である. 毎ステップの位置や速度を必ず近傍通信で送る必要はない. Unity 数値実験では, 環境が全状態を知っており, 各エージェントへ局所観測を返せる. 実機実験では, 安全のため外部 PC が全機位置を計測して各エージェントプロセスに配る構成が最初は望ましい. ただし, 論文上の「分散性」を明確にするため, 学習更新に用いる方策・乗数の通信は通信グラフ $\mathcal{G}_{\text{comm}}$ 上の近傍だけに制限する. この意味で, 本実験の分散性は「方策・乗数の更新と通信」に関するものであり, 訓練時の安全評価器や外部計測系が全状態を観測するかどうかとは分けて記述する.

7 事象駆動型コンセンサス primal-dual 学習アルゴリズム

7.1 重み行列とイベントトリガ

通信グラフ $\mathcal{G}_{\text{comm}}$ をサイクルとする場合、Otani 論文の五ロボット例に合わせて

$$w_{ii} = \frac{1}{9}, \quad w_{i,i^-} = w_{i,i^+} = \frac{4}{9}, \quad w_{ij} = 0 \quad (j \notin \{i, i^-, i^+\}) \quad (119)$$

とできる．一般の無向連結通信グラフでは，分散最適化で標準的な Metropolis 重み [10]

$$w_{ij} = \frac{1}{1 + \max\{|\mathcal{N}_i^{\text{comm}}|, |\mathcal{N}_j^{\text{comm}}|\}}, \quad \{i, j\} \in \mathcal{E}_{\text{comm}}, \quad w_{ii} = 1 - \sum_{j \in \mathcal{N}_i^{\text{comm}}} w_{ij} \quad (120)$$

を使うと実装しやすい．

以下の Algorithm 1 では，最後に受信した値の差分を現在値へ加える差分型の混合を用いる．この場合，自己重み w_{ii} は陽には現れず，オフ対角の混合係数

$$\omega_{ij}^{\text{mix}} = w_{ij}, \quad j \in \mathcal{N}_i^{\text{comm}} \quad (121)$$

だけが $\hat{\vartheta}_{j,q} - \hat{\vartheta}_{i,q}$ と $\hat{\mu}_{j,q} - \hat{\mu}_{i,q}$ に掛かる． W を凸結合として直接使う実装にしたい場合は， $z_{i,q} = \sum_{j \in \mathcal{N}_i^{\text{comm}} \cup \{i\}} w_{ij} \hat{\vartheta}_{j,q}$ のように書き換える必要がある．本稿では現在値 $\vartheta_{i,q}, \mu_{i,q}$ を起点にする差分型を採用する．

イベントトリガしきい値は，例えば

$$\epsilon_{i,q}^{\vartheta} = \epsilon_{\vartheta,0} \rho^q + \epsilon_{\vartheta,\min}, \quad \epsilon_{i,q}^{\mu} = \epsilon_{\mu,0} \rho^q + \epsilon_{\mu,\min}, \quad 0 < \rho < 1 \quad (122)$$

とする．理論上の可和性を意識する場合は $\epsilon_{\vartheta,\min} = \epsilon_{\mu,\min} = 0$ とし， $\rho < 1$ にする．実験では通信が極端に少なくなりすぎるのを避けるため，小さい下限値を入れてもよい．

7.2 ロールアウト勾配推定

学習イテレーション q で， N_{roll} 本のエピソードをサンプリングする．エージェント i の割引制約和を

$$\hat{J}_{i,q}^{C,(m)} = \kappa_{\gamma,H} \sum_{k=0}^{H-1} \gamma^k C_i(s_k^{(m)}, \mathbf{a}_k^{(m)}, s_{k+1}^{(m)}) \quad (123)$$

とする．双対更新方向は

$$\hat{h}_{i,q} = \frac{1}{N_{\text{roll}}} \sum_{m=1}^{N_{\text{roll}}} \left(\hat{J}_{i,q}^{C,(m)} - d_i \right) \quad (124)$$

である．これは μ_i の i 番目ブロックに入れ，他ブロックには 0 を入れる．

方策勾配は，実行 action $\mathbf{a}_{i,k}$ ではなく，方策分布からサンプルされた raw action $\tilde{\mathbf{a}}_{i,k}$ の log-probability に対して計算する．REINFORCE 型 [6, 5] または PPO 型 [7] で推定する．SMC Otani 論文 [1] の rollout-based variant に近い REINFORCE 型では，エージェント i の surrogate return を

$$\hat{R}_{i,k}^{(m)} = \sum_{\ell=k}^{H-1} \gamma^{\ell-k} \left(\frac{1}{N} \ell_i(s_{\ell}^{(m)}, \mathbf{a}_{\ell}^{(m)}, s_{\ell+1}^{(m)}) + \kappa_{\gamma,H} \left(\mu_{i,q}^{(i)} \right)^{\top} C_i(s_{\ell}^{(m)}, \mathbf{a}_{\ell}^{(m)}, s_{\ell+1}^{(m)}) \right) \quad (125)$$

とし, advantage を

$$\hat{A}_{i,k}^{(m)} = \hat{R}_{i,k}^{(m)} - V_{\xi_i}(o_{i,k}^{(m)}) \quad (126)$$

で与える. 正規化 advantage を $\tilde{A}_{i,k}^{(m)}$ とすると, コスト最小化の primal 方向は

$$\hat{g}_{i,q}^\ell = \frac{1}{N_{\text{roll}}} \sum_{m=1}^{N_{\text{roll}}} \sum_{k=0}^{H-1} \gamma^k \tilde{A}_{i,k}^{(m)} \nabla_{\vartheta} \log \pi_{\vartheta}(\tilde{\mathbf{a}}_{i,k}^{(m)} \mid o_{i,k}^{(m)}) \big|_{\vartheta=\vartheta_{i,q}} \quad (127)$$

である. この推定量は, エージェント i の局所行動確率に関する independent actor-critic 型の primal 方向であり, 理想問題 (107)–(108) の厳密な共有パラメータ勾配の不偏推定量とは限らない. 厳密な共有パラメータ勾配を求めるには, 他エージェントの score function も含める CTDE 型推定が必要になる. 本稿では, 分散実装しやすい局所方策勾配 surrogate として式 (127) を用いる. ここで $\tilde{A}_{i,k}^{(m)}$ は reward advantage ではなく **コスト advantage** であるため, Algorithm 1 では $-\eta_{\vartheta,q} \hat{g}_{i,q}$ により descent する. reward 最大化型の PPO/actor-critic 実装を流用する場合は, コストを reward の負値として扱うか, loss の符号を反転しないと, 更新方向が逆になる. エントロピー正則化を用いる場合は, スカラー advantage にベクトル勾配を足さず, 別項として

$$\hat{g}_{i,q}^{\text{ent}} = -\beta_{\text{ent},q} \frac{1}{N_{\text{roll}}} \sum_{m=1}^{N_{\text{roll}}} \sum_{k=0}^{H-1} \gamma^k \nabla_{\vartheta} \mathcal{H}(\pi_{\vartheta}(\cdot \mid o_{i,k}^{(m)})) \big|_{\vartheta=\vartheta_{i,q}} \quad (128)$$

を加える. したがって

$$\hat{g}_{i,q} = \hat{g}_{i,q}^\ell + \hat{g}_{i,q}^{\text{ent}} \quad (129)$$

を Algorithm 1 の primal 方向として用いる. この書き方では, $\hat{g}_{i,q}^\ell$ も $\hat{g}_{i,q}^{\text{ent}}$ も方策パラメータと同じ次元のベクトルであり, 次元的に整合している. 価値関数 V_{ξ_i} は

$$\min_{\xi_i} \frac{1}{2} \sum_{m,k} \left(V_{\xi_i}(o_{i,k}^{(m)}) - \hat{R}_{i,k}^{(m)} \right)^2 \quad (130)$$

を数ステップだけ Adam で更新する.

7.3 提案アルゴリズムの疑似コード

Algorithm 1 事象駆動型分散安全強化学習による Tello 協調取り囲み

Require: エージェント数 N , 通信グラフ $\mathcal{G}_{\text{comm}}$, 差分混合係数 ω_{ij}^{mix} , 取り囲み半径 R , 目標高度

h^{ref} , ステップサイズ $\eta_{\vartheta,q}, \eta_{\mu,q}$, しきい値 $\epsilon_{i,q}^{\vartheta}, \epsilon_{i,q}^{\mu}$.

- 1: 各 i は $\vartheta_{i,0} \in X$, 価値関数 $\xi_{i,0}$, $\mu_{i,0} \in M$ を初期化する.
- 2: $\hat{\vartheta}_{i,0} \leftarrow \vartheta_{i,0}$, $\hat{\mu}_{i,0} \leftarrow \mu_{i,0}$ とし, 初期値を近傍に送信する.
- 3: **for** $q = 0, 1, \dots, Q - 1$ **do**
- 4: 対象物軌道を生成または計測し, 式 (44) または (50) で各スロット $\mathbf{r}_{i,k}^{\text{slot}}$ を計算する.
- 5: 各エージェントは現在の方策 $\pi_{\vartheta_{i,q}}$ に基づき, Unity または実機で N_{roll} 本のオンポリシーロールアウトを収集する.
- 6: 各 i は式 (129) で primal 方向 $\hat{g}_{i,q}$, 式 (124) で dual 方向 $\hat{h}_{i,q}$ を計算する.
- 7: 各 i は近傍から最後に受信した $\hat{\vartheta}_{j,q}$, $\hat{\mu}_{j,q}$ を用いて混合点を計算する:

$$z_{i,q} = \vartheta_{i,q} + \sum_{j \in \mathcal{N}_i^{\text{comm}}} \omega_{ij}^{\text{mix}} (\hat{\vartheta}_{j,q} - \hat{\vartheta}_{i,q}),$$

$$z_{i,q}^{\mu} = \mu_{i,q} + \sum_{j \in \mathcal{N}_i^{\text{comm}}} \omega_{ij}^{\text{mix}} (\hat{\mu}_{j,q} - \hat{\mu}_{i,q}).$$

- 8: $E_i \hat{h}_{i,q}$ を, i 番目ブロックが $\hat{h}_{i,q}$ で他ブロックが 0 のベクトルとする.
- 9: 各 i は projected descent-ascent 更新を行う:

$$\vartheta_{i,q+1} = \Pi_X [z_{i,q} - \eta_{\vartheta,q} \hat{g}_{i,q}],$$

$$\mu_{i,q+1} = \Pi_M [z_{i,q}^{\mu} + \eta_{\mu,q} E_i \hat{h}_{i,q}].$$

- 10: 各 i は価値関数パラメータ ξ_i をローカルに更新する.
 - 11: **if** $\|\vartheta_{i,q+1} - \hat{\vartheta}_{i,q}\|_2 \geq \epsilon_{i,q}^{\vartheta}$ **or** $\|\mu_{i,q+1} - \hat{\mu}_{i,q}\|_2 \geq \epsilon_{i,q}^{\mu}$ **then**
 - 12: $\hat{\vartheta}_{i,q+1} \leftarrow \vartheta_{i,q+1}$, $\hat{\mu}_{i,q+1} \leftarrow \mu_{i,q+1}$ とし, 近傍 $j \in \mathcal{N}_i^{\text{comm}}$ に送信する.
 - 13: **else**
 - 14: $\hat{\vartheta}_{i,q+1} \leftarrow \hat{\vartheta}_{i,q}$, $\hat{\mu}_{i,q+1} \leftarrow \hat{\mu}_{i,q}$ とする. 通信しない.
 - 15: **end if**
 - 16: 目的値, 制約値, 取り囲み誤差, 通信回数を記録する.
 - 17: **end for**
-

■PPO 型に置き換える場合. REINFORCE 型は説明が簡単で Otani 論文の記述に近いが, 連続制御では PPO 型の方が安定することが多い. その場合も, イベントトリガ・コンセンサス・双対更新の外枠は同じであり, 式 (129) の代わりに clipped surrogate loss の勾配を $\hat{g}_{i,q}$ として用いる. ただし, PPO は clipped surrogate loss を最適化するため, 式 (117) の厳密な勾配方向ではなく, on-policy rollout に基づく安定化された surrogate direction として用いる. また, PPO は複数 epoch で同じロールアウトを再利用するため, 完全なオンポリシー性を保つには, 方策更新幅を小さくし, 古いデータを使い回しすぎないようにする. 実装上は, reward 最大化用の PPO loss をそのまま使うのではなく, コスト最小化

に合わせて advantage または loss の符号をそろえる.

8 数値実験設定

8.1 共通設定

最初の Unity 数値実験では, 実機移行を意識して以下の値を推奨する.

表 5 Unity 数値実験の推奨初期設定

項目	記号	値
エージェント数	N	シミュレーションでは 5, 実機初期検証では 3
飛行領域	Ω	$[-2.5, 2.5] \times [-2.5, 2.5] \times [0, 2.0]$ m
目標高度	h^{ref}	1.0 m
取り囲み半径	R	0.9 m ($N = 3$), 1.1 m ($N = 5$)
制御周期	Δt	0.1 s
エピソード長	H	150–250 step
割引率	γ	0.99
ロールアウト数	N_{roll}	4–16
方策ネットワーク	—	MLP: input–128–128–action
価値ネットワーク	—	MLP: input–128–128–1
primal stepsize	$\eta_{\vartheta, q}$	2×10^{-4} – 2×10^{-3} (PPO), または $10^{-3}/\sqrt{q+1}$ (REINFORCE)
dual stepsize	$\eta_{\mu, q}$	10^{-3} – 5×10^{-3}
乗数上限	μ_{max}	10^2 – 10^3
トリガ初期値	ϵ_0	0.02–0.1
トリガ減衰	ρ	0.999–0.99999

評価 episode は原則として horizon H まで実行し, 危険終了した場合は terminal penalty と done flag を記録する. 早期終了した episode の平均誤差を実際の長さ T_m で割ると危険終了した手法が過度に有利になることがあるため, 論文中の主要指標では分母を H に固定し, 終了後の不足区間には最終状態または penalty を入れる. 補助的に実行時間あたりの挙動を見たい場合だけ, T_m で正規化した値も併記する. 以下の $E_{\text{slot}}, E_R, E_{\text{edge}}, E_{\text{dir}}, E_{\text{lag}}$ は 1 本の評価 episode に対する値として定義する. 論文の図では, M_{eval} 本の評価 episode と複数乱数 seed に対して

$$\bar{E}(q) = \frac{1}{M_{\text{eval}}} \sum_{m=1}^{M_{\text{eval}}} E^{(m)}(q) \quad (131)$$

を表示し, 可能であれば標準偏差または標準誤差も併記する. 各学習イテレーション q で M_{eval} 本の評価 episode を実行する場合, 図には各 episode 指標の平均

$$\bar{E}_{\text{slot}}(q) = \frac{1}{M_{\text{eval}}} \sum_{m=1}^{M_{\text{eval}}} E_{\text{slot}}^{(m)}(q) \quad (132)$$

と標準偏差または標準誤差を示す. $E_R, E_{\text{edge}}, E_{\text{dir}}, E_{\text{lag}}$ についても同様に評価 episode 平均を用いる.

8.2 実験 1：静止対象物の正多角形取り囲み

対象物を領域中心に固定し、 $\mathbf{r}_{o,k} = 0$, $\mathbf{v}_{o,k} = 0$ とする。目標スロットは式 (44) で与える。この実験では、まず学習アルゴリズムが正多角形をフォーメーションできるかを確認する。

評価指標は次の通りである。

$$E_{\text{slot}}(q) = \frac{1}{NH} \sum_{i=1}^N \sum_{k=0}^{H-1} \|\mathbf{r}_{i,k} - \mathbf{r}_{i,k}^{\text{slot}}\|_2, \quad (133)$$

$$E_R(q) = \frac{1}{NH} \sum_{i=1}^N \sum_{k=0}^{H-1} \left| \|\mathbf{r}_{i,k} - \mathbf{r}_{o,k}\|_2 - R \right|, \quad (134)$$

$$E_{\text{edge}}(q) = \frac{1}{NH} \sum_{i=1}^N \sum_{k=0}^{H-1} \left| \|\mathbf{r}_{i,k} - \mathbf{r}_{i^+,k}\|_2 - \ell_{i,i^+,k}^{\text{edge,ref}} \right|. \quad (135)$$

比較手法は、(i) NSB 逆運動学のみ、(ii) 独立 PPO/REINFORCE、(iii) time-triggered consensus primal-dual、(iv) event-triggered consensus primal-dual とする。NSB 逆運動学のための baseline で E_{slot} まで比較する場合は、式 (78) のように低優先のスロット追従タスクを入れる。形状フォーメーションだけを比較する場合は、 E_R と E_{edge} を中心に評価し、スロット番号の対応を要求しないことを明記する。

8.3 実験 2：等速移動対象物の進行方向依存取り囲み

対象物を

$$\mathbf{r}_{o,k+1} = \mathbf{r}_{o,k} + \Delta t \bar{\mathbf{v}}_o, \quad \bar{\mathbf{v}}_o = [v_{\text{tar}}, 0]^\top \quad (136)$$

で動かす。領域境界に近づいた場合は折り返す、または円軌道へ切り替える。目標スロットは式 (50) で与える。この実験では、対象物の進行方向が変化したときに、ドローン群が衝突せず配置を変形できるかを確認する。

評価指標として、実験 1 の指標に加えて、進行方向依存配置誤差

$$E_{\text{dir}}(q) = \frac{1}{NH} \sum_{i=1}^N \sum_{k=0}^{H-1} \left\| \mathbf{r}_{i,k} - \mathbf{r}_{i,k}^{\text{slot,dir}} \right\|_2 \quad (137)$$

を用いる。ここで $\mathbf{r}_{i,k}^{\text{slot,dir}}$ は、式 (50) で得られる Case 2 の進行方向依存スロットを表す。さらに、対象物速度に対する重心追従遅れを

$$E_{\text{lag}}(q) = \frac{1}{H} \sum_{k=0}^{H-1} \left\| \bar{\mathbf{r}}_k - \mathbf{r}_k^{\text{cg,ref}} \right\|_2 \quad (138)$$

で評価する。

8.4 実験 3：円軌道・折れ線軌道の対象物取り囲み

対象物に次のような軌道を与える。

$$\text{円軌道: } \mathbf{r}_{o,k} = \begin{bmatrix} A \cos(\omega k \Delta t) \\ A \sin(\omega k \Delta t) \end{bmatrix}, \quad \text{折れ線軌道: } \mathbf{r}_{o,k+1} = \mathbf{r}_{o,k} + \Delta t \mathbf{v}_{o,k} \quad (139)$$

ただし、対象物軌道の内部モードを $\zeta_k^o = \iota(k) = \lfloor k/K_{\text{seg}} \rfloor \bmod 4$ として

$$\mathbf{v}_{o,k} = \begin{cases} [v, 0]^\top, & \iota(k) = 0 \\ [0, v]^\top, & \iota(k) = 1 \\ [-v, 0]^\top, & \iota(k) = 2 \\ [0, -v]^\top, & \iota(k) = 3 \end{cases} \quad (140)$$

を周期的に繰り返す。 K_{seg} は各直線区間の長さを表す制御ステップ数である。この ζ_k^o は式 (80) の対象物軌道内部状態に含める。CMDP 状態 (80) の ζ_k^o には、円軌道なら位相 $\omega k \Delta t$, 折れ線軌道なら区間番号 $\iota(k)$ と区間内ステップ数を入れる。これにより、同じ $\mathbf{r}_{o,k}, \mathbf{v}_{o,k}$ が観測されても、次の切替時刻が異なるケースを区別できる。この実験は、進行方向角 φ_k の急変に対する頑健性を調べるために使う。 φ_k のローパスフィルタ係数 λ_φ を変えて、取り囲み誤差と安全制約違反の trade-off を評価する。

8.5 実験 4：通信削減効果の評価

イベントトリガの通信削減効果は、Otani 論文と同様にノードレベル broadcast ratio で測る。ただし、本指標は、学習更新に用いる方策パラメータ ϑ_i と乗数スタック μ_i の Agent Process 間通信だけを測る。外部計測系からの状態配信、Unity 環境との入出力、安全監視通信、Tello Driver への RC 指令は含めない。

$$I_{i,q+1}^{\text{ET}} = \mathbf{1} \left\{ \left\| \vartheta_{i,q+1} - \hat{\vartheta}_{i,q} \right\|_2 \geq \epsilon_{i,q}^\vartheta \quad \text{or} \quad \left\| \mu_{i,q+1} - \hat{\mu}_{i,q} \right\|_2 \geq \epsilon_{i,q}^\mu \right\}, \quad (141)$$

$$r_{q+1}^{\text{ET}} = \frac{1}{N} \sum_{i=1}^N I_{i,q+1}^{\text{ET}}, \quad (142)$$

および

$$\bar{r}_Q^{\text{ET}} = \frac{1}{Q} \sum_{q=0}^{Q-1} r_{q+1}^{\text{ET}} \quad (143)$$

を記録する。time-triggered baseline は常に $r_q^{\text{TT}} = 1$ である。通信削減率は

$$1 - \bar{r}_Q^{\text{ET}} \quad (144)$$

である。さらに、実装上の通信量をより具体的に示すため、1 回の送信に必要なバイト数を

$$B_{\text{msg}} = n_{\text{byte}}^{\text{float}} (\dim \vartheta + \dim \mu) + B_{\text{header}} \quad (145)$$

とする。ここで $n_{\text{byte}}^{\text{float}} = 4$ なら float32, $n_{\text{byte}}^{\text{float}} = 8$ なら float64 を表す。旧記法は bfloat16 と紛らわしいため、本稿では用いない。1 回の送信で全近傍へ broadcast できる実装では、ノードレベル累積送信量を

$$B_{Q,\text{node}}^{\text{ET}} = \sum_{q=0}^{Q-1} \sum_{i=1}^N I_{i,q+1}^{\text{ET}} B_{\text{msg}}, \quad B_{Q,\text{node}}^{\text{TT}} = QN B_{\text{msg}} \quad (146)$$

とする．一方，Python socket など近傍ごとに unicast する実装では，近傍数を掛けた

$$B_{Q,\text{unicast}}^{\text{ET}} = \sum_{q=0}^{Q-1} \sum_{i=1}^N |\mathcal{N}_i^{\text{comm}}| I_{i,q+1}^{\text{ET}} B_{\text{msg}}, \quad (147)$$

$$B_{Q,\text{unicast}}^{\text{TT}} = Q \sum_{i=1}^N |\mathcal{N}_i^{\text{comm}}| B_{\text{msg}} \quad (148)$$

を併記する．したがって，論文中では broadcast ratio と node-level bytes を主指標とし，実装詳細を示す場合に unicast bytes を補助指標として報告する．

8.6 実験 5：Tello ダイナミクス・遅延・ノイズへの頑健性

Tello の実機移行を考えるため，次の domain randomization を入れる．

- 質量 m : $\pm 5\%$.
- 抗力係数 k_1, k_2, k_3 : $\pm 20\%$.
- 速度応答時定数 $\tau_v, \tau_z, \tau_\psi$: $\pm 30\%$.
- 通信遅延: 50–150 ms.
- 位置計測ノイズ: 標準偏差 1–3 cm.
- yaw 計測ノイズ: 標準偏差 1–3 deg.
- 風外乱: 水平加速度 0.05–0.15 m/s².

この実験で性能が大きく崩れる場合，(i) 方策行動を小さくする，(ii) コスト $w_{\Delta u}$ を大きくする，(iii) 安全シールドを強める，(iv) 逆運動学参照速度 w_v を大きくする，という順で調整する．

9 Unity と Python による数値実験の実現方法

9.1 推奨ソフトウェア構成

最初の実装では，次の 2 プロセス構成が分かりやすい．

- **Unity 側**: ドローン，対象物，障害物，壁，床，カメラを含む 3D シーンを管理し，物理シミュレーションまたはコマンドレベルモデルを進める．Python から行動を受け取り，観測・コスト・制約・終了フラグを返す．
- **Python 側**: PyTorch による方策・価値関数，事象駆動型 primal–dual 更新，ログ保存，描画，重み保存を担当する．

Unity–Python 間の通信は，ML-Agents，ZeroMQ，TCP socket，または ROS-TCP-Connector のどれでもよい．初学者には ML-Agents または ZeroMQ が扱いやすい．研究コードとして分散性を明確に見せたい場合は，Python 側で N 個の agent process を立て，それぞれが自分の ϑ_i, μ_i を持ち，socket で近傍通信を模擬するとよい．

9.2 Unity 側のクラス設計

Unity 側には次のクラスを用意する.

- **TelloAgent**: 各ドローンの状態, 行動適用, コマンドレベルモデルまたは 6 自由度モデルの更新を担当する.
- **TargetObject**: 対象物軌道を生成する. 静止, 等速, 円軌道, 折れ線軌道を切り替えられるようにする.
- **FormationManager**: 式 (44), (50) により各時刻の目標スロットを計算する.
- **SafetyEvaluator**: 式 (95)–(99) の制約コスト, 壁距離, 衝突判定を計算する.
- **PythonBridge**: Python へ観測を送り, Python から行動を受け取る.
- **Logger**: 軌跡, 制約値, 行動, 通信イベントを CSV または JSON で保存する.

9.3 Unity 1 ステップの処理

Algorithm 2 Unity 環境 1 ステップの処理

Require: 現在状態 s_k , Python から受け取った raw action $\tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k}$.

- 1: 各 raw action $\tilde{\mathbf{a}}_{i,k}$ を $[-1, 1]^4$ に clip し, 安全シールドを使う場合は executed action $\mathbf{a}_{i,k} = S_i(s_k, \tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k})$ へ補正する.
 - 2: 各 Tello に対して, コマンドレベルモデルまたは 6 自由度モデルで Δt だけ状態を進める.
 - 3: 対象物状態 $\mathbf{r}_{o,k}, \mathbf{v}_{o,k}$ を更新する.
 - 4: 目標スロット $\mathbf{r}_{i,k+1}^{\text{slot}}$ を計算する.
 - 5: 各 i について目的コスト $\ell_i(s_k, \mathbf{a}_k, s_{k+1})$ と制約コスト $C_i(s_k, \mathbf{a}_k, s_{k+1})$ を計算する.
 - 6: 各 i の局所観測 $o_{i,k+1}$ を構成する.
 - 7: 領域外逸脱, 衝突, バッテリ相当量, 時間上限に基づいて終了フラグを決める.
 - 8: $\{o_{i,k+1}, \ell_i, C_i, \text{done}\}_{i=1}^N$ を Python に返す.
-

9.4 Python 側の学習ループ

Python 側では, 以下のように書ける. ここでは概念が分かるように疑似コードで示す.

Algorithm 3 Python 側の学習ループ

```
1: 方策  $\pi_{\vartheta_i}$ , 価値関数  $V_{\xi_i}$ , 乗数  $\mu_i$ , 送信済み値  $\hat{\vartheta}_i, \hat{\mu}_i$  を初期化.
2: for  $q = 0, 1, \dots, Q - 1$  do
3:   各エージェントのバッファ  $\mathcal{D}_i \leftarrow \emptyset$ .
4:   for  $m = 1, \dots, N_{\text{roll}}$  do
5:     Unity 環境を reset する.
6:     for  $k = 0, 1, \dots, H - 1$  do
7:       for  $i = 1, 2, \dots, N$  do
8:         raw action  $\tilde{\mathbf{a}}_{i,k} \sim \pi_{\vartheta_i}(\cdot \mid o_{i,k})$  をサンプルし,  $\log \pi_{\vartheta_i}(\tilde{\mathbf{a}}_{i,k} \mid o_{i,k})$  を保存.
9:       end for
10:      raw action  $\tilde{\mathbf{a}}_{1,k}, \tilde{\mathbf{a}}_{2,k}, \dots, \tilde{\mathbf{a}}_{N,k}$  を Unity に送る. Unity 側で安全シールド後の executed
      action  $\mathbf{a}_{i,k}$  を決める.
11:      Unity から  $o_{i,k+1}, \ell_{i,k}, C_{i,k}, \text{done}$  を受け取る.
12:      各  $i$  のバッファ  $\mathcal{D}_i$  に  $(o_{i,k}, \tilde{\mathbf{a}}_{i,k}, \mathbf{a}_{i,k}, \ell_{i,k}, C_{i,k}, \log \pi_{i,k})$  を保存.
13:      if done then
14:        早期終了時刻  $T_m \leftarrow k + 1$  を記録し, 評価用には残り  $H - T_m$  ステップを最終状態
        または terminal penalty で補完する.
15:      break.
16:    end if
17:  end for
18: end for
19: 各  $i$  は  $\mathcal{D}_i$  から  $\hat{g}_{i,q}, \hat{h}_{i,q}$  を計算.
20: Algorithm 1 のコンセンサス primal-dual 更新とイベントトリガ通信を行う.
21: TensorBoard/CSV に  $F_i, G_i, E_{\text{slot}}, r_{q+1}^{\text{ET}}$  を保存.
22: end for
```

9.5 ログと図の出力

論文用の数値実験では, 次の図を出す.

1. 水平面軌跡図: 対象物軌道, ドローン軌道, 開始点, 終了点, 目標スロット, 安全半径を表示する.
2. 目的値曲線: $\frac{1}{N} \sum_i \hat{J}_i^\ell$ の移動平均.
3. 正規化制約曲線: $\max_i [\hat{J}_i^C]_\ell$ を制約成分ごとに表示し, しきい値 $[d_i]_\ell$ も描く. ここで $\hat{J}_i^C$ は式 (123) の正規化済み割引制約値である.
4. 取り囲み誤差: $E_{\text{slot}}, E_R, E_{\text{edge}}, E_{\text{lag}}$.
5. 通信量: r_q^{ET} , 累積通信削減率 $1 - \bar{r}_Q^{\text{ET}}$, および必要に応じて累積送信バイト数 $B_{Q,\text{node}}^{\text{ET}}$ または $B_{Q,\text{unicast}}^{\text{ET}}$.
6. 安全指標: 最小機体間距離, 最小壁距離, 高度最大偏差, 入力飽和率. 実機安全シールドを使う場合は, シールド前行動に対する C_i^{raw} と, シールド後行動に対する C_i^{shielded} を分けて記録する.

10 DJI Tello 実機実験の実現方法

10.1 実機実験での基本方針

実機では、制約付き強化学習の制約は「平均的な安全コストを小さくする仕組み」であり、瞬間的な衝突を必ず防ぐ保証ではない。したがって、実機実験では次の二重構造にする。

- **学習層**: SMC Otani 型のイベントトリガ primal-dual 更新で、取り囲み性能と制約コストを改善する。最初は Unity で学習し、実機では学習済み方策を用いる。
- **安全シールド層**: 実機制御時に、方策が出した行動をそのまま送らず、衝突・壁接近・高度逸脱を抑制するように補正する。非常停止条件では直ちに stop または land する。

安全シールドを用いる場合、5.1 節の記法では $\mathbf{a}_{i,k}^{\text{RL}} = \tilde{\mathbf{a}}_{i,k}$ であり、方策がサンプルした raw action $\mathbf{a}_{i,k}^{\text{RL}}$ に対する log-probability を方策勾配で用い、環境に実際に入る action は $\mathbf{a}_{i,k}^{\text{safe}} = \mathbf{a}_{i,k} = S(s_k, \mathbf{a}_{1,k}^{\text{RL}}, \dots, \mathbf{a}_{N,k}^{\text{RL}})$ とする。このとき、学習対象は「安全シールドを含む閉ループ環境」である。結果表示では、方策そのものの危険傾向を表す C_i^{raw} と、実際に実行された安全シールド後の C_i^{shielded} を分ける。双対更新に入れる制約値は、特に断らない限り、実際に実行された閉ループを評価する C_i^{shielded} とする。これは閉ループ系が安全シールドを含むためである。一方、raw policy 自体を安全側に寄せた場合は、 C_i^{raw} に対する補助ペナルティまたは補助制約を追加する。この区別を明記しないと、強い安全シールドにより raw policy が危険なままでも双対変数が大きくなり、という解釈上のずれが生じる。

本稿では、実機の主制御入力を SDK の rc コマンドに限定する。つまり、安全シールドが出力するのは最終的には $[-100, 100]^4$ の整数値であり、Tello Driver は rc a b c d という文字列だけを送る。go を使う場合も、Safety Supervisor が waypoint を安全範囲にクリップした後で go x y z speed を送るだけであり、低レベルの thrust, roll, pitch, motor speed は使わない。これにより、添付の Tello SDK User Guide に載っている簡易コマンドの範囲内で協調取り囲み実験を完結できる。

実機でオンライン学習を行う場合でも、最初は探索ノイズを小さくし、低高度・低速度・小さな飛行領域で行う。バッテリー 1 本あたりの飛行時間は限られるため、実機は主に評価・微調整に使い、大量の試行は Unity で行う。

10.2 推奨ハードウェア構成

- DJI Tello または Tello EDU を $N = 3$ 台から開始する。 $N = 5$ は Wi-Fi 混雑と安全確保が難しくなるため、シミュレーションで十分に安定した後に行う。複数台を同時制御する場合は、可能なら Tello EDU の swarm/AP モードを用いる。通常版 Tello では各機が独立した Wi-Fi AP になるため、複数 Wi-Fi インタフェース、ルータ構成、または順次接続制御が必要になる。初期段階では映像ストリームを停止し、外部計測系のみで位置を取得すると通信負荷を抑えられる。
- 外部計測系として、OptiTrack, Vicon, Qualisys, または天井カメラ + ArUco/AprilTag を用意する。Tello 単体の内部センサだけで複数機の相対位置を安定に得るのは難しい。
- 各 Tello に識別用マーカを付ける。対象物にも別のマーカを付ける。対象物は地上ロボット、手押し台車、または Unity/実機混合実験の仮想対象物でもよい。

- 制御 PC は Python/PyTorch を実行し、Tello SDK または ROS2/tello_ros 経由で各機へ指令を送る。
- 安全ネット、プロペラガード、広い屋内スペース、非常停止ボタンを用意する。

10.3 実機ソフトウェア構成

実機では、次の 4 種類のプロセスに分けるとデバッグしやすい。

1. **State Estimator:** 外部計測系から各 Tello と対象物の位置を受け取り、速度を推定する。速度は有限差分だけでなく、移動平均または Kalman filter で平滑化する。
2. **Agent Process:** エージェント i の方策推論、局所観測構成、学習時の v_i, μ_i 更新、近傍通信を担当する。
3. **Safety Supervisor:** 全機の状態を監視し、危険なら方策行動を上書きする。実機ではこのプロセスが最優先である。
4. **Tello Driver:** Tello SDK, DJITelloPy, または ROS2/tello_ros により、実際に `command`, `takeoff`, `land`, `stop`, `rc`, 必要に応じて `go` を送る。Driver は速度や waypoint を SDK テキストコマンドへ変換するだけで、低レベル姿勢・推力指令は扱わない。

実機で RC 指令値を用いる場合、 $u^{rc} = \text{round}(100a)$ はあくまで第一近似である。実験前に単機で $\pm 20, \pm 40, \pm 60$ など複数の指令を与え、外部計測系から得た平均速度により $v_{fb}^{\max}, v_{lr}^{\max}, v_{ud}^{\max}$ と符号規約を同定してから、Unity と実機のモデルを合わせる。

10.4 SDK コマンド層の実機実装

実機ソフトウェアでは、方策や安全シールドが返す値をそのまま Tello へ送らず、必ず SDK Command Layer を通す。この層の役割は次の 5 つである。

1. **状態遷移:** 実験開始時に各機へ `command`, `battery?`, 必要なら `streamoff` を送り、応答を確認してから `takeoff` する。終了時は `land`, 緊急時は `emergency` を送る。
2. **行動変換:** 方策・NSB・安全シールドの出力を、式 (35) により `rc a b c d` へ変換する。go モードでは、式 (39) により cm 単位の waypoint コマンドへ変換する。
3. **飽和と deadband:** 微小な速度指令は Tello の内部制御や床面条件で再現されにくいため、 $|u^{rc}| < u_{\text{dead}}$ なら 0 に丸める。一方、安全上の最大値 $u_{rc}^{\max, \text{real}}$ を超える値は clip する。
4. **keep-alive:** 制御周期で新しい行動が計算できない場合でも、安全が確認できる間は `rc 0 0 0 0` または `stop` を送る。一定時間コマンドがないと自動着陸する仕様があるため、通信監視と keep-alive を分けて実装する。
5. **応答監視:** `ok/error` 応答、状態パケット、バッテリー残量、Wi-Fi 品質、計測系の欠落を監視し、異常時は `stop`, `land`, 最後の手段として `emergency` へ移行する。

Algorithm 4 Tello SDK コマンド層：速度または waypoint から簡易コマンドへの変換

Require: モード $m \in \{\text{rc}, \text{go}\}$, 安全シールド後の水平速度 $\mathbf{u}_i^{xy, \text{safe}}$, 上下速度 $u_i^{z, \text{safe}}$, yaw 速度 $\omega_i^{\psi, \text{safe}}$, または waypoint 誤差 $\Delta \mathbf{p}_i^E$.

- 1: **if** $m = \text{rc}$ **then**
 - 2: 式 (31) で世界座標速度を機体座標の前後・左右速度へ変換する.
 - 3: 式 (35) で整数値 $u^{\text{rc}, \text{lr}}, u^{\text{rc}, \text{fb}}, u^{\text{rc}, \text{ud}}, u^{\text{rc}, \text{yaw}}$ を計算する.
 - 4: deadband と安全上限を適用する.
 - 5: 文字列 $\text{rc } u^{\text{rc}, \text{lr}} \ u^{\text{rc}, \text{fb}} \ u^{\text{rc}, \text{ud}} \ u^{\text{rc}, \text{yaw}}$ を送る.
 - 6: **else if** $m = \text{go}$ **then**
 - 7: 式 (36) で短距離 waypoint を作り, 機体座標系の cm 単位へ変換する.
 - 8: 式 (39) で $x^{\text{go}}, y^{\text{go}}, z^{\text{go}}$ を作り, 速度 s^{go} を 10–100 cm/s に clip する.
 - 9: **if** $|x^{\text{go}}|, |y^{\text{go}}|, |z^{\text{go}}|$ がすべて SDK の最小移動量未満, または保守的に $\|\Delta \mathbf{p}_i^E\| < 0.2 \text{ m}$ **then**
 - 10: **stop** を送る.
 - 11: **else**
 - 12: 文字列 $\text{go } x^{\text{go}} \ y^{\text{go}} \ z^{\text{go}} \ s^{\text{go}}$ を送る.
 - 13: **end if**
 - 14: **end if**
-

10.5 実機制御ループ

Algorithm 5 DJI Tello 実機での推論・制御ループ

- 1: 各 Tello と Wi-Fi/UDP で通信接続し, `command` を送って SDK mode に入り, `battery?` などで応答を確認する.
 - 2: 外部計測系を起動し, 座標系をキャリブレーションする.
 - 3: 各 Tello を `takeoff` させ, h^{ref} 付近で `rc 0 0 0 0` または `stop` によりホバリング状態へ移行させる.
 - 4: **for** 制御時刻 $k = 0, 1, 2, \dots$ **do**
 - 5: State Estimator が $\mathbf{r}_{i,k}, \mathbf{v}_{i,k}, h_{i,k}, \psi_{i,k}, \mathbf{r}_{o,k}, \mathbf{v}_{o,k}$ を更新する.
 - 6: Formation Manager が目標スロット $\mathbf{r}_{i,k}^{\text{slot}}$ と参照速度 $\mathbf{v}_{i,k}^{\text{ref}}$ を計算する.
 - 7: **for** $i = 1, 2, \dots, N$ **do**
 - 8: Agent Process i が局所観測 $o_{i,k}$ を構成し, 学習済み方策から行動 $\mathbf{a}_{i,k}^{\text{RL}}$ を計算する.
 - 9: **end for**
 - 10: Safety Supervisor が全行動 $\mathbf{a}_{1,k}^{\text{RL}}, \dots, \mathbf{a}_{N,k}^{\text{RL}}$ を検査し, 必要に応じて $\mathbf{a}_{i,k}^{\text{safe}}$ へ補正する.
 - 11: SDK Command Layer が $\mathbf{a}_{i,k}^{\text{safe}}$ または安全シールド後速度を `rc a b c d`, 必要に応じて `go x y z speed` へ変換し, 各 Tello に送信する.
 - 12: ログを保存する.
 - 13: **if** 最小機体間距離 $< d_{\text{emergency}}$, 計測喪失, 通信停止, 低バッテリー, 領域逸脱 **then**
 - 14: 全機に `stop` または `land` を送信し, 危険が継続する場合は `emergency` を送って実験を終了する.
 - 15: **end if**
 - 16: **end for**
-

10.6 安全シールドの例

Tello 実機は `rc` コマンドを内部速度制御器で処理するため, 本節の CBF-QP は推力や姿勢角を直接決める厳密な連続時間入力ではなく, `command-level` の安全フィルタとして用いる. したがって, 通信遅延, 速度飽和, `deadband`, および一次遅れがある実機では, 連続時間 CBF の形式的保証は近似的である. 実機実験では, 安全余裕を大きめに取り, `slack` 量, 非常停止距離, 計測欠落, 通信停止を監視して `stop/land/emergency` に切り替える.

簡単な安全シールドは, 速度コマンドを QP で補正する制御バリア関数 (CBF) 型の方法である [8]. 方策が出した水平速度を $\mathbf{u}_i^{\text{RL}} \in \mathbb{R}^2$ とし, 補正後水平速度を $\mathbf{u}_i \in \mathbb{R}^2$ とする. 機体間安全距離制約を

$$h_{ij}(\mathbf{r}) = \|\mathbf{r}_i - \mathbf{r}_j\|_2^2 - d_{\text{drone}}^2 \geq 0 \quad (149)$$

とすると, 連続時間 CBF 条件 $\dot{h}_{ij} + \alpha_h h_{ij} \geq 0$ は

$$2(\mathbf{r}_i - \mathbf{r}_j)^\top (\mathbf{u}_i - \mathbf{u}_j) + \alpha_h h_{ij}(\mathbf{r}) \geq 0 \quad (150)$$

となる. ここで α_h は s^{-1} の次元を持つ連続時間 CBF 係数であり, 制御周期が十分小さいときに各々

ンプリング時刻で課す。離散時間の一次近似として書くなら

$$h_{ij}(\mathbf{r}_{k+1}) \geq (1 - \alpha_h \Delta t) h_{ij}(\mathbf{r}_k) \quad (151)$$

である。この式を次時刻の安全集合 $h_{ij}(\mathbf{r}_{k+1}) \geq 0$ に直接結びつけるには、少なくとも $0 \leq \alpha_h \Delta t \leq 1$ とし、 $h_{ij}(\mathbf{r}_k) \geq 0$ から右辺が非負になるように設定する。 $\mathbf{r}_{k+1} \simeq \mathbf{r}_k + \Delta t \mathbf{u}_k$ を代入すると、式 (150) に Δt を掛けた同値な一次近似不等式が得られる。

壁制約では、壁からの単なる距離ではなく、安全マージンを引いた符号付き距離を用いる。壁からの距離を $d_i^{\text{wall}}(\mathbf{r}_i)$ 、壁安全距離を $d_{\text{wall}}^{\text{safe}}$ として

$$h_i^{\text{wall}}(\mathbf{r}_i) = d_i^{\text{wall}}(\mathbf{r}_i) - d_{\text{wall}}^{\text{safe}} \quad (152)$$

と定義すれば、 $h_i^{\text{wall}}(\mathbf{r}_i) \geq 0$ が「壁から少なくとも $d_{\text{wall}}^{\text{safe}}$ 離れている」ことを意味する。壁 CBF 条件は

$$\nabla h_i^{\text{wall}}(\mathbf{r}_i)^\top \mathbf{u}_i + \alpha_w h_i^{\text{wall}}(\mathbf{r}_i) \geq 0 \quad (153)$$

である。長方形領域では d_i^{wall} は四つの壁距離の最小値であり、角や複数壁が同時に近い点では微分不能になる。実装では、現在もっとも近い active wall の勾配を用いるか、smooth-min 近似により ∇h_i^{wall} を滑らかに計算する。実機では速度飽和、初期接近、計測ノイズにより QP が厳しくなりすぎる場合があるため、slack 付き QP として

$$\min_{\mathbf{u}, \delta} \sum_{i=1}^N \|\mathbf{u}_i - \mathbf{u}_i^{\text{RL}}\|_2^2 + \rho_\delta \left(\sum_{i < j} \delta_{ij}^2 + \sum_{i=1}^N (\delta_i^{\text{wall}})^2 \right) \quad (154)$$

$$\text{subject to } 2(\mathbf{r}_i - \mathbf{r}_j)^\top (\mathbf{u}_i - \mathbf{u}_j) + \alpha_h h_{ij}(\mathbf{r}) \geq -\delta_{ij}, \quad i < j, \quad (155)$$

$$\nabla h_i^{\text{wall}}(\mathbf{r}_i)^\top \mathbf{u}_i + \alpha_w h_i^{\text{wall}}(\mathbf{r}_i) \geq -\delta_i^{\text{wall}}, \quad i = 1, \dots, N, \quad (156)$$

$$\|\mathbf{u}_i\|_\infty \leq u_{\text{max}}^{\text{real}}, \quad i = 1, \dots, N, \quad (157)$$

$$\delta_{ij} \geq 0, \quad \delta_i^{\text{wall}} \geq 0 \quad (158)$$

を用いる。 ρ_δ は十分大きく設定し、slack が大きい場合は安全余裕が不足している警告としてログに残す。高度方向については、方策が出した上下速度を $u_{i,z}^{\text{RL}}$ 、補正後を $u_{i,z}$ とし、別の 1 次元 QP

$$\min_{u_z, \delta^z} \sum_{i=1}^N |u_{i,z} - u_{i,z}^{\text{RL}}|^2 + \rho_z \sum_{i=1}^N (\delta_i^z)^2 \quad (159)$$

$$\text{subject to } u_{i,z} \leq \alpha_z (h_{\text{max}}^{\text{real}} - h_i) + \delta_i^z, \quad (160)$$

$$-u_{i,z} \leq \alpha_z (h_i - h_{\text{min}}^{\text{real}}) + \delta_i^z, \quad (161)$$

$$|u_{i,z}| \leq u_{z,\text{max}}^{\text{real}}, \quad \delta_i^z \geq 0, \quad i = 1, \dots, N \quad (162)$$

を課すか、同等の saturation を必ず入れる。これにより、10.1 節で述べた衝突・壁接近・高度逸脱の三種類を実機安全シールドで扱う。ただし、これは計測遅延、速度飽和、QP 解の品質が許す範囲で危険接近を抑える防護層であり、任意条件下の絶対的安全保証として主張しない。QP infeasible, slack が所定しきい値以上、計測欠落、通信停止、最小距離 $d_{\text{emergency}}$ 未満のいずれかが発生した場合は、直ちに stop または land に切り替える。

QP ソルバを入れない最初の実装では、危険距離に入ったら単純に

$$\mathbf{u}_i^{\text{safe}} = \text{sat}_{u_{\text{max}}^{\text{real}}} \left(\mathbf{u}_i^{\text{RL}} + K_{\text{rep}} \sum_{j \neq i} \frac{\mathbf{r}_i - \mathbf{r}_j}{\|\mathbf{r}_i - \mathbf{r}_j\|_2^2 + \epsilon} + K_{\text{wall}} \nabla d_i^{\text{wall}} \right) \quad (163)$$

とし、方策入力に反発速度を加えたうえで速度を小さく saturate するだけでもよい。ここで $\text{sat}_{u_{\max}^{\text{real}}}(\cdot)$ は、各成分またはノルムを実機速度上限 $u_{\max}^{\text{real}}$ 以内に切り詰める飽和写像である。 ∇d_i^{wall} も、壁距離が非微分な点では最も近い壁の外向き法線または smooth-min 近似の勾配として実装する。ただし、論文実験としては QP 型の方が説明しやすい。

10.7 段階的な実機移行手順

1. **単機ホバリング**: 1 台だけで takeoff, hover, land, 外部計測, RC 指令, ログ保存を確認する。
2. **単機スロット追従**: 対象物は固定し, 1 台の Tello が円周上の 1 点を追従する。
3. **2 台の安全距離検証**: 2 台で一定距離を保つ。安全シールドが設定した条件で作動し, 危険時には stop/land へ移行することをログで確認する。
4. **3 台の静止対象物取り囲み**: 学習済み方策または NSB 参照速度で正三角形をフォーメーションする。
5. **3 台の低速移動対象物取り囲み**: 対象物を 0.1–0.2 m/s 程度で動かす。
6. **イベントトリガ通信の導入**: 各 Agent Process 間の ϑ_i, μ_i 通信をソフトウェア上でイベントトリガ化し, 通信ログを保存する。
7. **実機微調整**: Unity で学習した方策を固定して実機評価し, 必要なら低探索ノイズで短時間の fine-tuning を行う。

11 実装時に特に注意すべき点

11.1 Otani 論文の理論保証と本実験の関係

SMC Otani 論文の有限時間解析は exact gradient を前提としており, 本稿のような Tello/Unity 実験ではロールアウトに基づく確率勾配を用いる。さらに, 式 (127) は共通パラメータ問題の厳密な方策勾配ではなく, 分散実装しやすい independent actor-critic 型の局所 surrogate direction である。したがって, 実験結果として「Otani 論文の定理がそのまま実機で成立した」とは主張しない。正しくは,

Otani 論文の事象駆動型コンセンサス primal-dual 構造を, 有限ホライズン・ロールアウトに基づく局所 surrogate gradient へ置き換えた実装版として用い, 通信量削減と empirical constraint cost の低下を検証する。

と述べる。

11.2 分散性の説明

実機では安全のため 1 台の PC が全機の位置を知っている構成から始めることが多い。この場合でも, 学習アルゴリズムとしての分散性は次のように説明できる。

- 方策パラメータ ϑ_i と乗数スタック μ_i は各 Agent Process がローカルに保持する。
- primal-dual 更新において, 他エージェントのロールアウトデータや勾配そのものは共有しない。
- 近傍から受け取るのは, イベントトリガで送られた $\hat{\vartheta}_j, \hat{\mu}_j$ のみであり, 通信は $\mathcal{G}_{\text{comm}}$ 上で行う。

- 状態計測, Unity のコスト評価, 実機の Safety Supervisor は安全監視と局所観測生成のためのインフラであり, 学習更新の中央集約とは区別する.

より強い分散性を示したい場合は, 各 Tello に対応する小型 PC または別プロセスを用意し, 対象物位置と自機位置だけを受け取り, 近傍相対位置は通信で取得するようにする. また, 制約コストも全ペア最小距離ではなく $C_{i,1}^{\text{local}}$ のような近傍安全グラフ上の制約に置き換える.

11.3 Tello 実機での通信と制御周期

Tello SDK は, コマンド送信用, 状態受信用, 映像受信用に異なる UDP ポートを用いる. 実装では, コマンド送信用 socket と状態受信用 socket を分け, 状態受信スレッドは常時動かす. 実機では, rc 指令を 10–20 Hz 程度で送り, 計測が一時的に欠けた場合でも直ちに stop/land へ移行できるようにする. 学習イテレーション q はエピソード単位で進むため, リアルタイム制御周期 k と混同しないようにする.

- 制御ステップ k : 0.05–0.1 秒ごとに rc a b c d, または低レートで go x y z speed を送る実時間ループ.
- 学習イテレーション q : 1 つまたは複数のエピソードが終わった後に方策・乗数を更新するループ.
- イベントトリガ通信: 学習イテレーション q の更新後に ϑ_i, μ_i を送るかどうかを判定する通信. これは Tello SDK の飛行コマンドとは別の Agent Process 間通信である.
- keep-alive: 新しい制御入力計算できない短時間には rc 0 0 0 0 または stop を送る. ただし, 計測喪失・通信停止・低バッテリーでは keep-alive で粘らず land へ移行する.

go モードでは, 各コマンドの完了待ち時間と応答遅れが制御性能を支配するため, 移動対象物速度を小さくし, waypoint 周期を長めにする. 一方, rc モードでは細かいフィードバックが可能だが, UDP パケットの欠落や Wi-Fi 混雑に弱い. したがって, 論文実験では, Unity で rc コマンド層を含む遅延・飽和・deadband を再現し, 実機では低速・小領域・外部計測・安全シールド付きで検証する.

12 最小実装チェックリスト

1. Unity で N 台の Tello 風オブジェクトと対象物を生成できる.
2. Python から N 台分の行動を送り, Unity から N 台分の観測・コスト・制約を返せる.
3. 式 (44) と (50) の目標スロットが正しく描画できる.
4. NSB またはスロット追従速度だけで安全に取り囲みができる.
5. 独立方策で目的コストが下がる.
6. primal-dual 更新を入れると制約コストが下がる.
7. コンセンサス混合を入れても発散しない.
8. イベントトリガを入れて通信回数が減る.
9. domain randomization 後も極端に性能が落ちない.
10. 実機 1 台で command, takeoff, rc 0 0 0 0, stop, land の状態遷移と低速移動が安定する.
11. 実機 3 台で安全シールドの作動条件, slack, stop/land/emergency 切替ログを確認できる.
12. 実機 3 台で rc モードにより静止対象物の取り囲みを達成できる. 余裕があれば go モードの

waypoint 追従でも比較する.

13 まとめ

本稿では, SMC Otani 論文の事象駆動型分散安全強化学習アルゴリズムを, Uesaka–Namerikawa 型の協調取り囲み問題に適用し, DJI Tello を用いた Unity 数値実験および実機実験へ展開するための具体的な定式化を与えた. ポイントは, 取り囲み制御を明示的に設計し, それを強化学習の目的コスト・制約コスト・参照速度に組み込むことである. これにより, 純粋な探索だけに頼らず, 逆運動学的に妥当な行動を出発点として, 安全制約を primal–dual 更新で学習できる.

実装面では, Unity で十分に学習・検証した後, Tello 実機では学習済み方策を低速で実行し, 外部計測系と安全シールドで瞬間的な危険接近を抑制しながら段階的に移行することが重要である. 特に本稿では, 最終的な実機入力を `rc a b c d, go x y z speed, stop, takeoff, land` などの Tello SDK 簡易コマンドに限定することで, Tello の実際の制御インタフェースと矛盾しない実験設計にした. この構成により, 通信量削減, 取り囲み性能, 安全制約の三つを同時に評価でき, SMC Otani 論文のアルゴリズムを実ロボット応用に拡張する数値実験・実機実験として説得力のある形にできる.

参考文献

- [1] S. Otani, N. Hayashi, and M. Inuiguchi, “Decentralized Safe Reinforcement Learning with Event-Triggered Communication,” submitted manuscript / SMC-related manuscript, 2026.
- [2] Y. Uesaka and T. Namerikawa, “Inverse Kinematics based Cooperative Target-enclosing Behavior for Swarms of Vehicles,” *Proceedings of the Japan Society of Mechanical Engineers, Motion and Vibration Control Symposium*, pp. 208–213, 2009.
- [3] Ryze Tech, *Tello SDK 2.0 User Guide*, Version 1.0, 2018.
- [4] W. Giernacki, J. Rao, S. Sladic, A. Bondyra, M. Retinger, and T. Espinoza-Fraire, “DJI Tello Quadrotor as a Platform for Research and Education in Mobile Robotics and Control Engineering,” *Proc. 2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 735–744, 2022.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [6] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [8] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [9] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” *Advances in Neural Information Processing Systems*, 2017.

- [10] A. Nedić, A. Ozdaglar, and P. A. Parrilo, “Constrained consensus and optimization in multi-agent networks,” *IEEE Transactions on Automatic Control*, vol. 55, no. 4, pp. 922–938, 2010.